

Semantic Communication in 6G: ML/DL Implementation

Arpit Raj

The LNM Institute of Information Technology (LNMIIT)
Jaipur, India

Abstract—This research paper deals with next-gen semantic communication. As wireless networks approach the theoretical bounds of classical Shannon capacity, next-generation telecommunications (6G) must transition from exact bit-stream transmission to goal-oriented, semantic communication. This research presents an end-to-end software simulation of a semantic communication physical layer, demonstrating the inherent error resilience of natural language processing (NLP) embeddings over noisy wireless channels. Rather than mapping text to binary sequences, the proposed deterministic framework utilizes dense vector representations (e.g., Sentence-BERT) as joint source-channel coders. These high-dimensional mathematical representations of human intent are transmitted through a simulated Additive White Gaussian Noise (AWGN) channel under variable Signal-to-Noise Ratio (SNR) conditions. At the receiver, decoding is achieved not through classical binary error-correction, but via cosine similarity mapping against a localized vector knowledge base.

I. CONVENTIONAL COMMUNICATION

Conventional communication systems are built on Claude Shannon's information theory. In these architectures, the message is transmitted bit by bit.

A. Source Encoder

The primary step in the communication flow is the source encoder. The goal here is to transform information into a source binary code while using as few bits as possible. A prominent method to achieve this and reduce redundancy is Huffman coding.

II. HUFFMAN CODING ANALYSIS

A. Fixed-Length Coding (Without Huffman)

To illustrate the necessity of Huffman coding, let us first consider a scenario without it, assuming every symbol uses a fixed length of 2 bits.

TABLE I
FIXED-LENGTH CODING PROBABILITIES

Symbol	Probability	Code
A	0.5	00
B	0.25	01
C	0.125	10
D	0.125	11

For a transmission of 100 symbols, the total number of bits required would be:

$$\text{Bits} = (50 \times 2) + (25 \times 2) + (12.5 \times 2) + (12.5 \times 2) \quad (1)$$

$$\text{Total Bits} = 200 \text{ bits} \quad (2)$$

B. Variable-Length Coding (With Huffman)

Huffman coding dictates that for more frequent symbols, the number of bits should be less. A fundamental rule for unique decoding is that no code starts with another complete code (prefix rule).

TABLE II
HUFFMAN CODING PROBABILITIES

Symbol	Probability	Code
A	0.5	0
B	0.25	10
C	0.125	110
D	0.125	111

For a 100-symbol message using Huffman coding, the total number of bits is calculated as:

$$\text{Bits} = (50 \times 1) + (25 \times 2) + (12.5 \times 3) + (12.5 \times 3) \quad (3)$$

$$\text{Total Bits} = 175 \text{ bits} \quad (4)$$

By utilizing this method, a total of **25 bits are saved** compared to the fixed-length approach.

III. INFORMATION THEORY METRICS

A. Entropy

Entropy gives the minimum average bits required per symbol. It is defined mathematically as:

$$H(X) = -\sum p_i \log_2(p_i) \quad (5)$$

For example, if two symbols A and B each have a probability $p = 0.5$, then the entropy is:

$$H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1 \text{ bit/symbol} \quad (6)$$

B. Average Code Length

The average code length (L) is calculated by summing the products of the probabilities and their respective code lengths:

$$L = \sum p_i l_i \quad (7)$$

Using the Huffman coding example provided in Section II-B:

$$L = (0.5 \times 1) + (0.25 \times 2) + (0.125 \times 3) + (0.125 \times 3) \quad (8)$$

$$L = 1.75 \text{ bits/symbol} \quad (9)$$

C. Efficiency and Redundancy

The efficiency (η) of the coding scheme evaluates how close the average code length is to the theoretical minimum (entropy):

$$\eta = \frac{H}{L} \times 100 \quad (10)$$

Finally, the redundancy (R) of the system is defined as:

$$R = 1 - \frac{H}{L} \quad (11)$$

IV. CHANNEL ENCODING AND FORWARD ERROR CORRECTION (FEC)

When data travels through a communication channel, inherent noise can corrupt the transmitted bits. For instance, a transmitted sequence of 1011001 might be received as 1010001 due to interference.

To mitigate this, a channel encoder introduces redundancy. Forward Error Correction (FEC) algorithms send extra redundant bits alongside the data, allowing the receiver to not only detect but also correct errors without requiring a retransmission.

A. Error Mitigation Mechanisms

A primitive example of FEC is repetition coding, where a bit is transmitted multiple times (e.g., transmitting 111 instead of 1). If noise corrupts the signal to 101, a majority voting logic at the receiver will correctly identify the intended bit as 1.

Modern systems categorize these mechanisms into two domains:

- **Error Detection:** Techniques like Parity bits, Checksums, and Cyclic Redundancy Checks (CRC).
- **Error Correction:** Implementations such as Hamming codes, Reed-Solomon codes, and Convolutional codes.

The foundational metric for these codes is the **Hamming Distance**, defined as the number of bit positions in which two codewords differ. For example, the distance between 1011 and 1001 is exactly 1.

B. Detection and Correction Capabilities

The capabilities of a block code depend strictly on its minimum Hamming distance, d_{min} .

- **Error Detection Capability:** A code can detect up to $d_{min} - 1$ errors.
- **Error Correction Capability (t):** A code can correct up to:

$$t = \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor \quad (12)$$

As an example, if $d_{min} = 3$, the system can detect $(3-1) = 2$ errors and can correct $\lfloor \frac{2}{2} \rfloor = 1$ error.

C. Hamming Code Implementation

The Hamming code functions by adding parity bits at positions corresponding to powers of two (e.g., positions 1, 2, 4, 8). This structure inherently identifies the exact position of a corrupted bit.

For m data bits and r parity bits, the constraint for successful error correction is:

$$2^r \geq m + r + 1 \quad (13)$$

Assuming a message length of $m = 4$ bits, we evaluate the inequality: $2^3 = 8 \geq 4 + 3 + 1 = 8$. Therefore, $r = 3$. A 4-bit message requires exactly 3 parity bits.

V. DIGITAL MODULATION TECHNIQUES

Raw binary streams (e.g., 101101001) cannot be efficiently transmitted over physical channels directly. They must be mapped to continuous analog waveforms.

A. Quadrature Amplitude Modulation (QAM)

Quadrature Amplitude Modulation (QAM) achieves high spectral efficiency by utilizing two sinusoidal carriers of the same frequency that are exactly 90° out of phase:

- The In-phase (I) carrier: represented by $\cos(2\pi f_c t)$
- The Quadrature (Q) carrier: represented by $\sin(2\pi f_c t)$

By modulating both the amplitude and the phase, each unique (I, Q) coordinate locates a specific point in a constellation diagram, allowing multiple bits to be transmitted per symbol.

For a 64-QAM system, there are 64 distinct points in the constellation grid ($64 = 2^6$). Therefore, each symbol encapsulates exactly $\log_2(64) = 6$ bits of information.

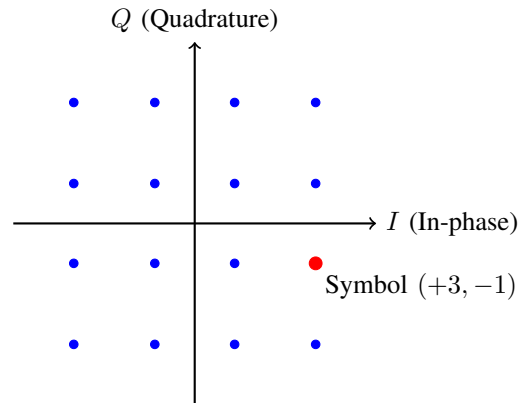


Fig. 1. Visualization of a QAM Constellation Grid mapping I and Q values.

B. Transmission and Symbol Mapping

In an operational QAM transmitter, an incoming bitstream (e.g., 110101 001011 111000) is segmented into groups. Each group maps to a single constellation point defined by a predetermined mapping table.

Assuming a specific data segment 101011 maps to the coordinates $I = +3$ and $Q = -1$, the transmitted signal $s(t)$ is formulated as:

$$s(t) = I \cos(2\pi f_c t) + Q \sin(2\pi f_c t) \quad (14)$$

Different bit combinations produce distinct waveforms, ensuring unique symbols are transmitted across the channel.

C. Signal Extraction at the Receiver

At the receiver, the goal is to map the incoming waveform back to a constellation point by extracting the I and Q values.

- 1) **I-Extraction:** The received signal is multiplied by $\cos(2\pi f_c t)$ and passed through an integrator (averaging filter). Because sine and cosine are orthogonal over a full period, this process nullifies the Q component and isolates the I magnitude.
- 2) **Q-Extraction:** The signal is multiplied by $\sin(2\pi f_c t)$ and averaged, which eliminates the I component and isolates Q .

Due to channel noise, the extracted values will rarely be perfect integers. Suppose the processing yields $I \approx 3.1$ and $Q \approx -0.8$. A decision boundary algorithm maps this back to the nearest valid constellation point $(3, -1)$, which is then demodulated back into the bit sequence 101011.

D. Adaptive Modulation and Noise Trade-offs

Higher-order modulations exponentially increase the data rate, but suffer from increased susceptibility to noise.

TABLE III
MODULATION SCHEMES AND DATA RATES

Modulation	Symbols	Bits / Symbol
BPSK	2	1
QPSK	4	2
16-QAM	16	4
64-QAM	64	6
256-QAM	256	8

In robust schemes like QPSK, the constellation points are physically separated by larger Euclidean distances, meaning the channel noise must be exceptionally high to force a symbol into an adjacent decision region. Conversely, in dense constellations like 64-QAM or 256-QAM, the points are packed closely together, making them highly sensitive to interference.

To optimize throughput while maintaining acceptable Bit Error Rates (BER), modern transceivers employ **Adaptive Modulation**. If the Signal-to-Noise Ratio (SNR) drops, the system dynamically downshifts the modulation order (e.g., 64-QAM $\rightarrow$ 16-QAM $\rightarrow$ QPSK) to prioritize connection stability over peak data rate.

VI. THE BANDWIDTH CRUNCH AND THE PARADIGM SHIFT

As modern wireless networks transition toward 6G, data demands are scaling exponentially. Emergent technologies such as autonomous vehicles, Augmented Reality/Virtual Reality

(AR/VR), and digital twins require an unprecedented volume of data transmission. However, physical bandwidth remains strictly limited, and transmitting every individual bit across the network has become unsustainably expensive.

Conventional systems are fundamentally bound by the necessity to transmit raw data streams, attempting to reconstruct the exact binary sequence at the receiver side. Yet, modern network data contains immense structural redundancy. While traditional communication insists on sending every single bit with absolute numerical accuracy, the actual semantic meaning or core intent of the message is often orders of magnitude smaller than the raw bitstream representation.

To demonstrate the boundaries of classical physical layer scaling, Table IV outlines the performance metrics across frequency ranges (FR1 and FR2) alongside historical baselines. Despite massive expansions in bandwidth and the deployment of higher subcarrier spacing (SCS), the spectral efficiency (bit/s/Hz) fundamentally saturates. This bottleneck underscores the necessity of a architectural shift from bit-level replication to semantic preservation.

VII. ARCHITECTURE OF A SEMANTIC COMMUNICATION SYSTEM

To bypass the limitations of Shannon's traditional framework, a semantic layer is integrated directly into the transceiving pipeline. Instead of translating raw sequences to bits, the system extracts the vital objects, actions, deep intent, meaning, and contextual background of the data, transmitting exclusively this compressed semantic essence.

A. End-to-End System Block Diagram

The operational framework replaces standard source components with intelligent deep learning blocks that interface directly with localized and global knowledge bases.

Modern implementations of these semantic components utilize advanced artificial intelligence architectures, such as Transformers, BERT, and deep Autoencoders. These models are uniquely capable of learning highly dense, low-dimensional mathematical representations of human language and environment states.

B. Paradigm Comparison Matrix

The shift from classical communication to semantic architecture changes the fundamental metrics used to evaluate system performance, as contrasted in Table V.

VIII. SHARED KNOWLEDGE BASES AND GRAPH REPRESENTATIONS

A. Knowledge Base Synchronization

A foundational requirement for semantic communication is the synchronization of knowledge. The transmitting agent (T_x) and receiving agent (R_x) must share a structurally consistent understanding of the application domain prior to transmission. This relationship is governed by the shared knowledge base constraint:

$$\text{SKB}_{T_x} \approx \text{SKB}_{R_x} \quad (15)$$

TABLE IV
PHYSICAL LAYER THROUGHPUT AND EFFICIENCY PERFORMANCE METRICS

Frequency Range	SCS	Bandwidth	DL Throughput	UL Throughput	Efficiency DL	Efficiency UL
FR1	15 kHz	50 MHz	288.9 Mbit/s	309.1 Mbit/s	5.78 bit/s/Hz	6.18 bit/s/Hz
FR1	30 kHz	100 MHz	584.3 Mbit/s	625.0 Mbit/s	5.84 bit/s/Hz	6.25 bit/s/Hz
FR1	60 kHz	100 MHz	577.8 Mbit/s	618.1 Mbit/s	5.78 bit/s/Hz	6.18 bit/s/Hz
FR2	60 kHz	200 MHz	1.08 Gbit/s	1.18 Gbit/s	5.40 bit/s/Hz	5.90 bit/s/Hz
FR2	120 kHz	400 MHz	2.15 Gbit/s	2.37 Gbit/s	5.38 bit/s/Hz	5.93 bit/s/Hz
Comparison with LTE						
All bands	15 kHz	20 MHz	100 Mbit/s	100 Mbit/s	5.00 bit/s/Hz	5.00 bit/s/Hz
Comparison with 3G						
All bands	-	5 MHz	21.1 Mbit/s	17.2 Mbit/s	4.22 bit/s/Hz	3.44 bit/s/Hz

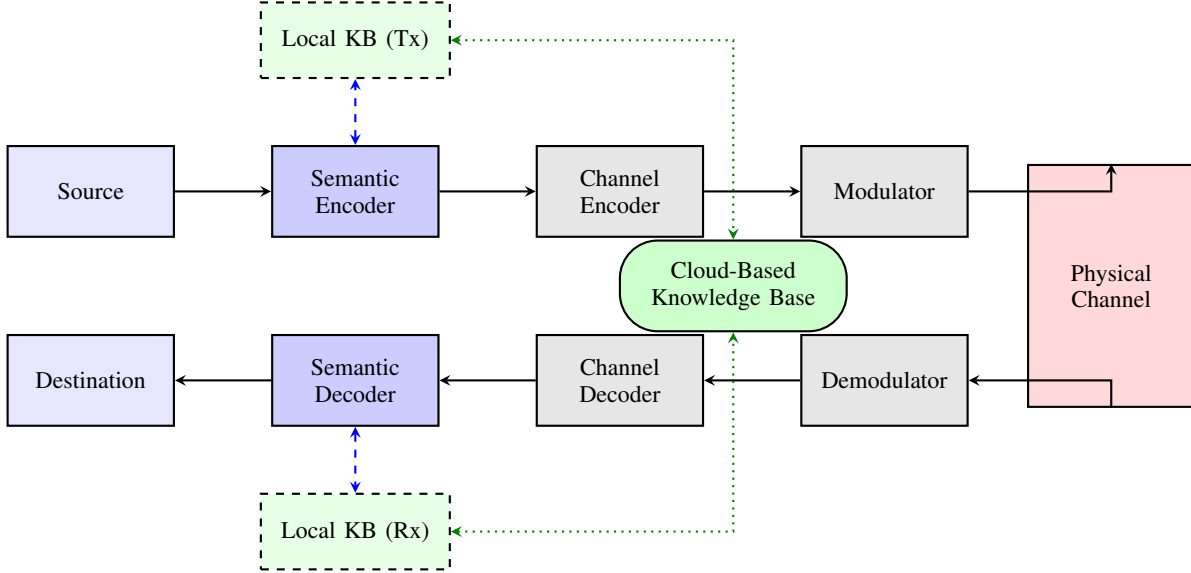


Fig. 2. System architecture of an intelligent semantic communication framework utilizing synchronized shared knowledge bases.

TABLE V
SYSTEM METRICS AND PARADIGMS COMPARISON

Parameter	Traditional Communication	Semantic Communication
Transmission Core	Raw binary bits	Underlying data meaning
System Focus	Structural accuracy of sequence	Realized intent of message
Primary Metric	Bit Error Rate (BER)	Semantic similarity index
Theoretical Basis	Shannon Information Theory	AI-driven Semantic Theory
Context Treatment	Meaning + Context + Redundancy	Meaning alone

Information within a knowledge graph is formally structured into discrete relational triplets:

$$\text{Triplet} = (\text{Entity}_1, \text{Relation}, \text{Entity}_2) \quad (16)$$

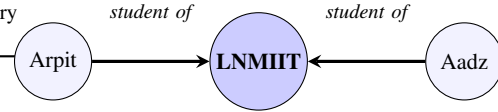


Fig. 3. A sample structural knowledge graph demonstrating entity-relation mapping.

The Shared Knowledge Base (SKB) acts as a comprehensive common repository, encapsulating domain-specific ontologies, structured knowledge graphs, explicit localized facts, semantic text embeddings, contextual relationship maps between diverse entities, and pre-trained language model parameters.

B. Knowledge Graph-Based Representations

Semantic communication frameworks rely heavily on highly structured semantic representations. A knowledge graph organizes real-world entities and maps the explicit relationships connecting them, translating disjoint data points into a cohesive relational network that provides deep situational context.

IX. SEMANTIC ENCODING AND DECODING MECHANICS

A. Semantic Encoding

Instead of treating text as a stream of characters to be processed by standard source encoders, an NLP module parses raw text sequences into distinct entities, relations, and attributes. These relational triplets are then mapped directly into dense vector spaces via deep learning models.

The resulting high-dimensional semantic vector captures the contextual intent using significantly fewer total bits than would be required if the original sentence were translated into text characters and expanded using traditional error-correcting block codes.

B. Semantic Decoding

Upon receiving the channel-degraded vector embedding, the receiver does not execute bit-by-bit error correction. Instead, the decoding block combines the incoming semantic vector with its localized Shared Knowledge Base (SKB_{Rx}) to feed an intelligent language generation model.

While the syntactical structure of the newly generated sentence may visually differ from the original input text, the semantic content, core message, and human intent are preserved perfectly.

X. CHANNEL NOISE RESILIENCE AND PROBABILISTIC INFERENCE

A. Traditional Error Overhead

In a traditional physical layer setup, when a channel disturbance corrupts transmitted bits, the data packets fail their respective integrity checks (e.g., CRC). Consequently, a retransmission request is generated across the link via Automatic Repeat Request (ARQ) protocols. This introduces significant system vulnerabilities:

- Substantial increases in overall processing latency.
- Higher instantaneous channel bandwidth consumption.
- Pronounced degradation in end-to-end network efficiency.

B. Semantic Error Mitigation via Inference

Conversely, semantic communication frameworks demonstrate innate resilience against channel noise. If the received vector representation of a knowledge graph triplet suffers corruption during transmission, the receiver can leverage the strong structural priors embedded within its localized knowledge base (SKB_{Rx}).

Consider an illustrative scenario where an edge agent transmits engineering context, and a key node in the semantic graph sequence is corrupted:

$$(\text{VLSI, subject of, ? [Corrupted]}) \quad (17)$$

Because the synchronized knowledge base contains deep contextual domain mapping, the receiver does not request a retransmission. Instead, it applies probabilistic reasoning across its existing network paths to evaluate the missing structural component, immediately inferring that the corrupted term refers to *Signals/Systems*.

Errors are resolved through native receiver-side **Inference** rather than channel-dependent **Retransmission**, estimating the lost information accurately without requiring additional channel resources. This framework provides a viable pathway for ultra-reliable, low-latency execution across 6G networks, Edge AI systems, and autonomous vehicle arrays.

XI. MACHINE LEARNING AND DEEP LEARNING INTEGRATION IN SEMANTIC COMMUNICATION

A. Transmitting Meaning Rather Than Bits

In traditional communication, information is represented as symbols and bits, and performance is analyzed using metrics such as Bit Error Rate (BER) and Signal-to-Noise Ratio (SNR). The shift towards semantic communication introduces a new paradigm: transmitting meaning rather than bits.

However, this introduces a significant challenge: semantic meaning is highly complex, context-dependent, and subjective. There is no standard mathematical expression for “meaning.” Designing an algorithm to extract semantic components—such as object, action, intent, and relation—is extremely difficult. For unstructured data like images, videos, and audio, this complexity further increases. Determining which component is semantically important for transmission is not straightforward. Hardcoded, rule-based mathematical models are impractical. This limitation strongly motivates the integration of Machine Learning (ML) and Deep Learning (DL) techniques.

B. Deep Learning as a Semantic Encoder

Neural networks trained on large datasets replace manually designed feature extraction. The objective is to learn a compressed representation. The semantic encoder receives raw data (such as text, speech, image, and video) and transforms it:

$$Z = f_{\theta}(X) \quad (18)$$

Where:

- X is the raw input data.
- f_{θ} is a neural network parameterized by weights θ .
- Z is a semantic embedding containing the most relevant information.

The transmitter sends this learned semantic representation Z , ensuring that the core meaning is preserved across the transmission.

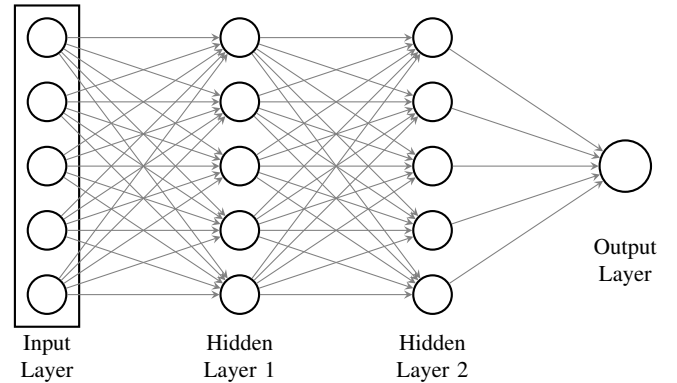


Fig. 4. Neural network acting as a semantic encoder transforming input data into an output embedding.

C. Neural Networks in Semantic Communication

A neural network is a computational model that learns a mapping between inputs and outputs by adjusting internal parameters through training. The learned transformation can be expressed as:

$$y = f(x) \quad (19)$$

1) **Mathematical Model of a Neuron:** Consider an artificial neuron receiving multiple inputs: $x_1, x_2, x_3, \dots, x_n$. Each input has an associated weight: $w_1, w_2, w_3, \dots, w_n$. The neuron computes the weighted sum:

$$z = x_1 w_1 + x_2 w_2 + \dots + x_n w_n + b \quad (20)$$

Expressed in vector notation:

$$z = w^T x + b \quad (21)$$

The result is then passed to an activation function to produce the output a :

$$a = \phi(z) = \phi(w^T x + b) \quad (22)$$

Example Calculation:

Given inputs $x = [2, 3]$, weights $w = [0.5, 0.8]$, and bias $b = 1$:

$$z = (2)(0.5) + (3)(0.8) + 1 = 1 + 2.4 + 1 = 4.4 \quad (23)$$

Applying the ReLU activation function $\phi(z) = \max(0, z)$:

$$a = \max(0, 4.4) = 4.4 \quad (24)$$

This final value a represents the neuron output.

D. Role of Activation Function

Without an activation function, a neural network is just a large linear equation. Real-world semantic relationships are non-linear. The activation function introduces this vital non-linearity into the system. Common activation functions include:

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$, where $0 < \sigma(x) < 1$
- **Tanh:** $\tanh(x)$, where $-1 < \tanh(x) < 1$
- **ReLU:** $\text{ReLU}(x) = \max(0, x)$

E. Layer Organization and Feature Abstraction

Neurons are organized into distinct layers to process information hierarchically:

- **Input Layer:** Receives raw information.
- **Hidden Layers:** Perform feature extraction and abstraction. Lower layers learn simple patterns, while higher layers learn complex concepts.
- **Output Layer:** Produces semantic embeddings, classifications, or predictions.

For example, in text processing:

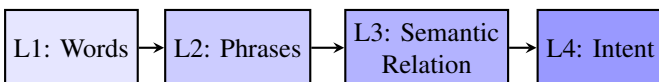


Fig. 5. Hierarchical abstraction of text across neural network layers.

F. Training and Gradient Descent

Initially, the network's weights are random. The network gradually improves by comparing its predictions against correct answers. The objective is to minimize the Loss Function $L(y, \hat{y})$.

Through Gradient Descent, the network adjusts weights to reduce this loss:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w} \quad (25)$$

Where η represents the learning rate. This process is repeated millions of times during training. Eventually, the network discovers useful semantic patterns hidden within the data. After passing through multiple layers, the network transforms raw data into a compact numerical representation known as an embedding:

$$Z = [z_1, z_2, \dots, z_n] \quad (26)$$

G. The Semantic Communication Pipeline

In this ML-integrated architecture, the standard transmitter and receiver modules are replaced by semantic counterparts.

- **Semantic Encoder:** Maps the message to an embedding (Message $\rightarrow$ Embedding).
- **Semantic Decoder:** Maps the embedding back to meaning (Embedding $\rightarrow$ Meaning).

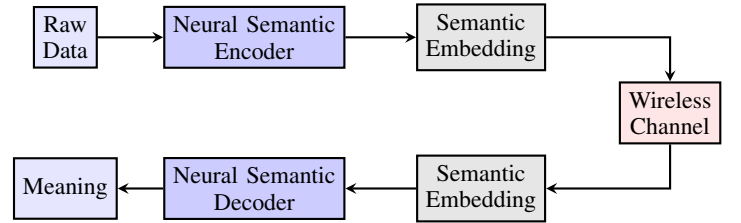


Fig. 6. End-to-End Semantic Communication Pipeline using Neural Networks.

XII. NUMERICAL ANALYSIS OF DEEP LEARNING MODELS

A. Forward Propagation with a Single Hidden Neuron

To understand how semantic representations are learned, it is essential to trace the mathematical operations during a forward pass. Consider a simple neural network with an input layer of 2 neurons and an output layer of 1 neuron, utilizing a single hidden neuron with a Rectified Linear Unit (ReLU) activation function.

Let the inputs be $x_1 = 2$ and $x_2 = 3$, with a true target value of $y = 10$. The network initializes with the following random weights and biases:

- Hidden layer weights: $w_1 = 0.5, w_2 = 0.8$
- Hidden layer bias: $b_h = 1$
- Output layer weight: $v = 1.2$
- Output layer bias: $b_o = 0$

The pre-activation value at the hidden neuron, z_h , is computed as:

$$z_h = w_1x_1 + w_2x_2 + b_h \quad (27)$$

$$z_h = (0.5)(2) + (0.8)(3) + 1 \quad (28)$$

$$z_h = 1 + 2.4 + 1 = 4.4 \quad (29)$$

Applying the ReLU activation function, $a_h = \max(0, z_h)$:

$$a_h = \max(0, 4.4) = 4.4 \quad (30)$$

Moving to the output neuron, the predicted value $\hat{y}$ is computed:

$$\hat{y} = va_h + b_o \quad (31)$$

$$\hat{y} = (1.2)(4.4) + 0 = 5.28 \quad (32)$$

The error in this prediction is measured using the Mean Squared Error (MSE) loss function:

$$L = \frac{1}{2}(y - \hat{y})^2 \quad (33)$$

$$L = \frac{1}{2}(10 - 5.28)^2 = 11.1392 \quad (34)$$

B. Backpropagation and Weight Optimization

To improve the network's prediction, the weights must be updated using gradient descent. The first step is to calculate the gradient of the loss with respect to the output prediction:

$$\frac{\partial L}{\partial \hat{y}} = (\hat{y} - y) = (5.28 - 10) = -4.72 \quad (35)$$

The negative sign indicates that the current prediction is too low compared to the true value.

Next, we update the output weight v . The gradient is computed via the chain rule:

$$\frac{\partial L}{\partial v} = (\hat{y} - y)a_h = (-4.72)(4.4) = -20.768 \quad (36)$$

Given a learning rate $\eta = 0.01$, the new output weight is:

$$v_{new} = v - \eta \frac{\partial L}{\partial v} \quad (37)$$

$$v_{new} = 1.2 - 0.01(-20.768) = 1.40768 \quad (38)$$

Similarly, we update the hidden weights w_1 and w_2 . The gradient for w_1 requires passing the error through the ReLU derivative:

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y) \times v \times \text{ReLU}'(z_h) \times x_1 \quad (39)$$

Since $z_h = 4.4 > 0$, the derivative $\text{ReLU}'(4.4) = 1$.

$$\frac{\partial L}{\partial w_1} = (-4.72)(1.2)(1)(2) = -11.328 \quad (40)$$

The updated weight w_1 becomes:

$$w_1 = 0.5 - 0.01(-11.328) = 0.61328 \quad (41)$$

Following the same process for w_2 :

$$\frac{\partial L}{\partial w_2} = (-4.72)(1.2)(1)(3) = -16.992 \quad (42)$$

$$w_2 = 0.8 - 0.01(-16.992) = 0.96992 \quad (43)$$

C. Iterative Semantic Learning

After a single forward and backward pass, the network parameters have shifted, resulting in a prediction much closer to the target value.

Before Training	After 1 Iteration
$w_1 = 0.5$	$w_1 \approx 0.61$
$w_2 = 0.8$	$w_2 \approx 0.96$
$v = 1.2$	$v \approx 1.4$
$a_h = 4.4$	$a_h = 5.13$
$\hat{y} = 5.28$	$\hat{y} = 7.23 \rightarrow \text{Closer to } 10$

TABLE VI
PARAMETER ADJUSTMENTS AFTER ONE CYCLE OF GRADIENT DESCENT.

After millions of training examples consisting of forward passes, loss computations, backpropagation, and weight updates, the network eventually learns abstract semantic relationships, such as recognizing that "car" $\approx$ "vehicle" and "fast" $\approx$ "speed".

XIII. TRANSFORMER ARCHITECTURES IN SEMANTIC COMMUNICATION

A. The Self-Attention Mechanism

While traditional recurrent neural networks process data sequentially, Transformer architectures introduce the concept of self-attention. This allows neural networks to capture long-range semantic relationships within data. Unlike recurrent networks, a transformer can simultaneously analyze all components of an input sequence and determine their relative importance.

For an input sequence $X = \{x_1, x_2, \dots, x_n\}$, the model computes the mathematical relationship between every possible pair of elements. The resulting attention weights dictate which parts of the input contribute most significantly to the overall semantic meaning of the message. For example, in the sentence "neural networks is a great concept of machine learning," the attention mechanism strongly connects "neural networks" to "machine learning."

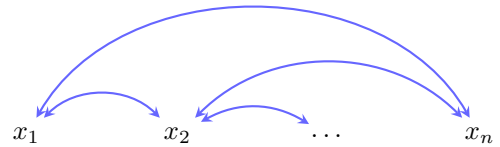


Fig. 7. Self-attention mechanism computing pairwise semantic relationships simultaneously across the entire input sequence.

B. Generative AI for Semantic Reconstruction

In modern semantic frameworks, Generative AI plays a critical role at the receiver end. The receiver utilizes a semantic decoder g_ϕ to reconstruct the original intent of the message from the transmitted semantic embedding Z . The reconstructed message $\hat{X}$ is formulated as:

$$\hat{X} = g_\phi(Z) \quad (44)$$

In this generative process, the exact wordings may differ from the original input, but the core context and meaning remain identical.

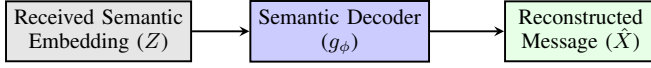


Fig. 8. Semantic reconstruction utilizing Generative AI at the receiver.

XIV. PERFORMANCE METRICS IN SEMANTIC SYSTEMS

A. Inadequacy of Traditional Metrics

In standard communication frameworks, performance is heavily evaluated based on exact signal reconstruction metrics. These include:

- **Bit Error Rate (BER):** $\frac{\text{Number of incorrect bits}}{\text{Total number of bits}}$
- **Symbol Error Rate (SER):** $\frac{\text{Incorrect symbols}}{\text{Total symbols}}$
- **Packet Error Rate (PER):** $\frac{\text{Corrupted packets}}{\text{Total packets}}$

These metrics fundamentally assume that absolute exactness in signal reconstruction is required. However, semantic communication allows for syntactic modification. As a result, metrics like BER may indicate a significant error or failure even when the fundamental meaning of the transmitted message remains completely unchanged.

B. Semantic Evaluation: The BLEU Score

To effectively measure semantic preservation, metrics from natural language processing are adapted. One prominent metric is the BLEU (Bilingual Evaluation Understudy) score.

The BLEU score measures the overlap between the generated reconstructed text and the original reference text using n -gram matching techniques. A higher score indicates a stronger similarity between the strings.

$$\text{BLEU} \in [0, 1] \quad (45)$$

While BLEU is a step forward, it primarily evaluates literal word similarity and n -gram overlaps. Consequently, it may still fail to capture deeper semantic equivalence where entirely different vocabulary is used to perfectly convey the same meaning, highlighting the ongoing need for advanced semantic evaluation metrics.

XV. SEMANTIC EVALUATION METRICS IN NLP

As semantic communication shifts the focus from bit-level accuracy to intent preservation, evaluating the performance of these systems requires specialized metrics derived from Natural Language Processing (NLP). This chapter outlines the foundational methodologies for quantifying semantic similarity and translation quality.

A. N-gram Matching

An n -gram is defined as a contiguous sequence of n items (typically words) extracted from a given text sequence. They are fundamental in analyzing the structural composition of sentences.

- **Unigrams** ($n = 1$): Capture the raw vocabulary of the text.
- **Higher-order N-grams** ($n > 1$): Capture sentence structure and localized context. The higher the order of the n -gram, the better it captures semantic relations.

Given a sample sentence: "I lost my mind, no one noticed", the extraction yields:

- **Unigram** ($n = 1$): {I, lost, my, mind, no, one}
- **Bigram** ($n = 2$): {I lost, lost my, my mind, mind no}
- **Trigram** ($n = 3$): {I lost my, lost my mind, ...}

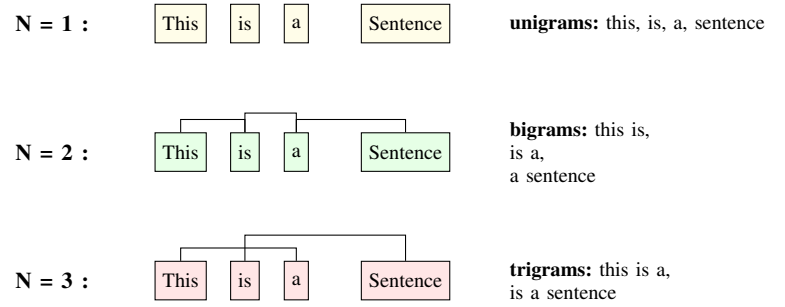


Fig. 9. Visualization of continuous N-gram sequencing applied to a sample sentence.

B. BLEU Score Evaluation

The Bilingual Evaluation Understudy (BLEU) score primarily measures *precision* by calculating how many generated phrases appear sequentially in the reference text.

The BLEU metric is formulated as:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (46)$$

Where:

- $N = 4$ (Standard maximum n -gram limit)
- w_n = Weighting factor
- p_n = n -gram precision

1) **Brevity Penalty (BP):** A critical component of BLEU is the Brevity Penalty, which prevents the system from unfairly rewarding extremely short, high-precision generations that miss broader semantic context. Consider the following:

- **Reference:** *the college is good*
- **Generated:** *the college*

Although the precision is exceptionally high, vital information is omitted. To combat this, the penalty is calculated as:

$$\text{BP} = e^{(1 - \frac{r}{c})} \quad (47)$$

where c is the candidate length and r is the reference length.

2) Advantages and Limitations:

TABLE VII
BLEU SCORE CHARACTERISTICS

Advantages	Limitations
Easy and efficient to compute Highly effective for machine translation	Primarily measures word overlap Fails to capture deeper semantic meaning

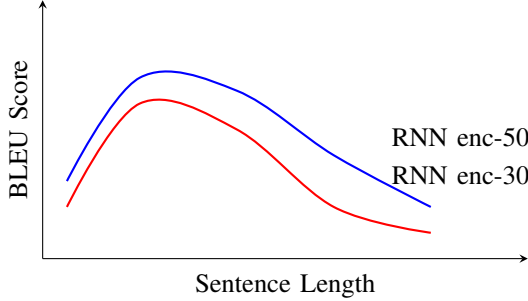


Fig. 10. Effect of sequence length on BLEU performance across varying encoder parameters.

C. ROUGE Score

In contrast to BLEU's precision focus, ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is fundamentally recall-oriented. It evaluates how much of the original reference information successfully appears within the generated text.

1) **ROUGE-N**: This metric measures the raw matching of n -grams between the candidate and the reference:

$$\text{ROUGE}_N = \frac{\text{matching } n\text{-grams}}{\text{reference } n\text{-grams}} \quad (48)$$

2) **ROUGE-L**: ROUGE-L leverages the Longest Common Subsequence (LCS) to score the text. Instead of relying on consecutive matches, it identifies the longest co-occurring sequence of words. This approach natively rewards the structural maintenance of the sequence without requiring strict, contiguous n -gram overlap.

XVI. TRANSFORMERS FOUNDATION IN SEMANTIC COMMUNICATION

A. Evaluation Metrics: BLEU vs ROGUE

In evaluating generated textual representations within semantic structures, it is essential to distinguish between the primary metrics utilized for scoring.

TABLE VIII
COMPARISON OF BLEU AND ROGUE METRICS

Attribute	BLEU	ROGUE
Focus	Precision	Recall
Popular in Denominator	Translation	Summarization
Penalizes extra words	Generated text words	Reference text words
Penalizes missing words	✓	×
	×	✓

B. Limitations of Previous Architectures

Before the advent of transformers, classical architectures faced significant limitations in handling sequence data and semantic continuity:

- **Feedforward Neural Networks (FFN)**: An FFN treats every input independently. Given an input vector $x = [x_1, x_2, x_3, x_4]$, there is no mechanism to understand word order, context, or long-range dependency.
- **Recurrent Neural Networks (RNN)**: RNNs introduced memory into the architecture:

$$h_t = f(x_t, h_{t-1}) \quad (49)$$

where x_t is the current word and h_t is the hidden state. While the hidden state successfully carried information forward, long sentences ultimately lost information due to the vanishing gradient problem.

Transformers eliminate recurrence entirely. In a transformer framework, every word directly examines every other word—a mechanism fundamentally known as *Self-Attention*.

C. Input Processing and Positional Encoding

In a transformer, each word is converted into a vector. Because transformers process all words simultaneously, they inherently lack sequential awareness and require explicit position information.

Positional Encoding (PE) is injected using sine and cosine functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (50)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (51)$$

where d represents the embedding dimension. The final embedding fed into the network is the combination of these factors:

$$\text{Embedding} = \text{word vector} + \text{position} \quad (52)$$

D. The Self-Attention Mechanism

Every embedded word generates three distinct vectors via learnable weight matrices (W_Q, W_K, W_V):

$$\text{Query (Q)} = XW_Q \quad (53)$$

$$\text{Key (K)} = XW_K \quad (54)$$

$$\text{Value (V)} = XW_V \quad (55)$$

To understand this conceptually through an analogy:

- **Query**: The text you type in a search bar.
- **Key**: Titles, tags, and metadata of all candidate videos.
- **Value**: The actual video you watch, once a match is found.

The core interaction is determined by the **Attention Score**, calculated as QK^T . For example, given $Q = [2, 1]$ and $K = [3, 2]$, the score is $QK^T = (2)(3) + (1)(2)$.

1) **Scaling and Softmaxing**: Calculating the raw attention score alone can result in a large vector with unstable values. To mitigate this, the score undergoes **Scaling**:

$$\text{Scaled Score} = \frac{QK^T}{\sqrt{d_k}} \quad (56)$$

where d_k is the key dimension.

Following scaling, **Softmaxing** is applied to convert the scores into normalized probabilities:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}} \quad (57)$$

This ensures that $\sum_i \text{attention} = 1$. For example, an unnormalized score array of $[8, 2, 1]$ becomes $[0.995, 0.004, 0.001]$, clearly indicating that the first word holds the vast majority of the importance.

The final output is the attention map multiplied by the Value (V):

$$Z = \sum \alpha_i V_i \quad (58)$$

The complete, unified attention equation is formalized as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (59)$$

E. Multi-Head Attention

A single attention mechanism focuses on only one relationship. Transformers, therefore, use multiple attention heads. Different heads learn different semantic relations across the text sequence.

$$\text{Head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (60)$$

$$\text{MultiHead} = \text{Concat}(\text{Head}_1, \dots, \text{Head}_n) \quad (61)$$

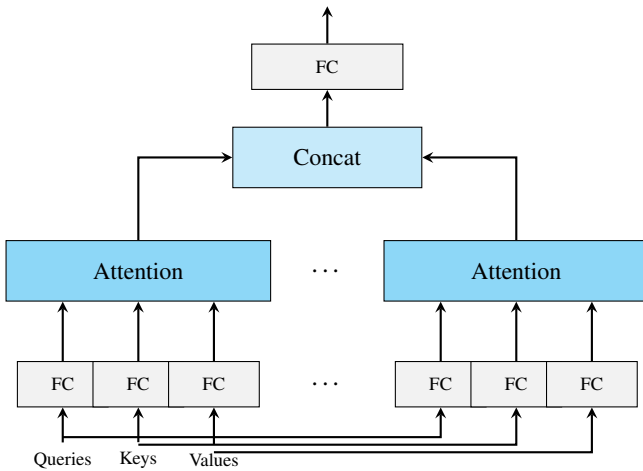


Fig. 11. Structural workflow of Multi-Head Attention, utilizing multiple linear projections (FC) and parallel Attention computation blocks.

F. Feed-Forward Networks and Layer Normalization

After processing through the attention mechanism, each token passes through a Feed-Forward Network (FFN) to allow non-linear semantic transformation:

$$\text{FFN}(x) = W_2(\text{ReLU}(W_1x + b_1)) + b_2 \quad (62)$$

To ensure training stability and to prevent exploding activations within the deep network, Layer Normalization (x) is applied:

$$\text{LayerNorm} = \frac{(x - \mu)}{\sqrt{\sigma^2 + \epsilon}} \quad (63)$$

G. Cosine Similarity

In semantic vector spaces, similarity is calculated using Cosine Similarity, computing the normalized dot product of two vectors, $A = [a_1, a_2, \dots, a_n]$ and $B = [b_1, b_2, \dots, b_n]$:

$$\text{CosSim} = \frac{A \cdot B}{|A||B|} \quad (64)$$

where the dot product and magnitude are derived as:

$$A \cdot B = \sum a_i b_i \quad (65)$$

$$|A| = \sqrt{\sum a_i^2} \quad (66)$$

XVII. TRANSFORMER ARCHITECTURE AND DEEP LEARNING INTEGRATION

This chapter details the fundamental deep learning structures utilized in the semantic processing of information, specifically focusing on the Transformer architecture and its core component, the Multi-Head Attention mechanism.

A. The Transformer Architecture

The Transformer framework relies heavily on self-attention mechanisms to process sequential data, discarding the need for recurrent structures. It consists of an encoder block that processes the input sequence and a decoder block that generates the output sequence.

B. Multi-Head Attention Mechanism

The Multi-Head Attention component resolves long-range dependencies by allowing the model to jointly attend to information from disparate representation subspaces across different sequential positions. The process formally segments the input Queries (Q), Keys (K), and Values (V) into multiple parallel computation heads before evaluating the attention weights and systematically recombining the output.

XVIII. TRANSFORMER ARCHITECTURE FOR SEMANTIC COMMUNICATION

A. Step 1: Input Embedding

The first step maps discrete input tokens into continuous dense vectors.

$$x_i \in \mathbb{R}^{d_{model}} \quad (67)$$

For example, an input representation could be: $x = [0.7, 0.2, -0.3, 0.9]$.

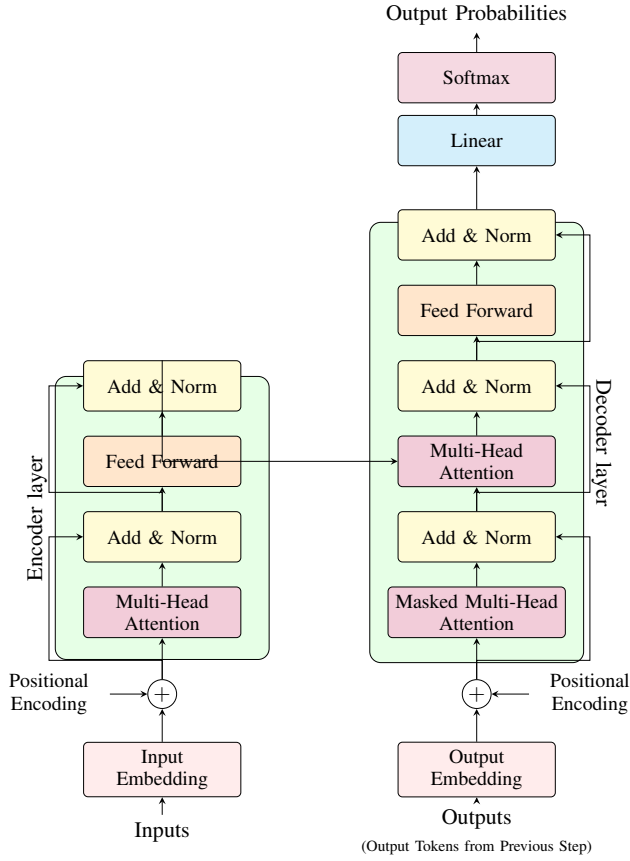


Fig. 12. Systematic representation of the Transformer Architecture, illustrating the interconnected Encoder and Decoder pipelines.

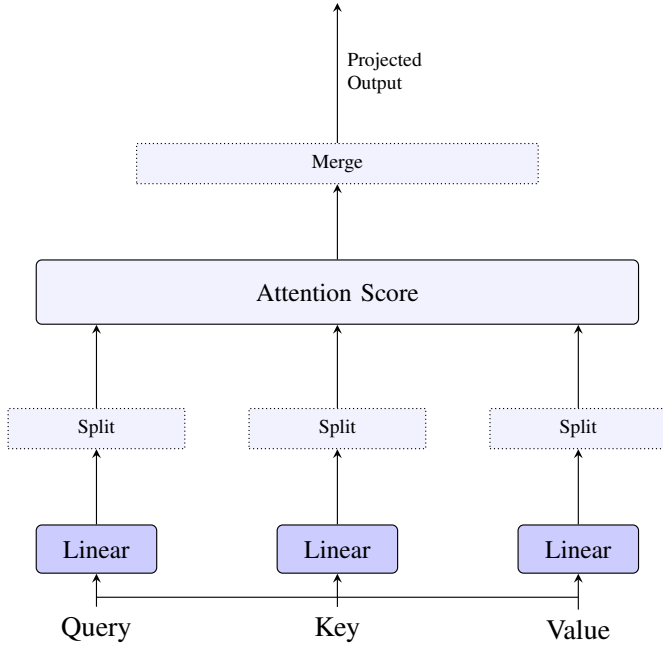


Fig. 13. Detailed schematic of the Multi-Head Attention Mechanism, illustrating the segmentation and subsequent merging of the Q, K, and V tensors.

B. Step 2: Positional Encoding

To inject sequence order information into the model, positional encodings are added to the input embeddings.

$$E_i = x_i + PE_i \quad (68)$$

Where the positional encoding (PE) is defined using sine and cosine functions of different frequencies:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (69)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (70)$$

C. Step 3: Encoder Layer

The encoder runs N times. Each encoder layer has a fundamental repeating structure: *Multi-Head Attention* $\rightarrow$ *Add & Normalize* $\rightarrow$ *Feed Forward* $\rightarrow$ *Add & Normalize*.

D. Step 4: Self Multi-Head Attention

The self-attention mechanism derives the Query (Q), Key (K), and Value (V) matrices from the embedded input X .

$$X = [E_1, E_2, \dots, E_n] \quad (71)$$

$$Q = XW_Q \quad (72)$$

$$K = XW_K \quad (73)$$

$$V = XW_V \quad (74)$$

Suppose $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$. This makes Q, K, V vectors by the formulas. The attention score between token i and token j is:

$$\text{score}_{ij} = Q_i K_j^T \quad (75)$$

This measures relevance using the concept of multi-head attention. Let $i = 2$ and Q_2 , then compare with every key vector K ($K_1, K_2, K_3, K_4, K_5, K_6$). Compute all scores:

$$Q_2 K_1^T, Q_2 K_2^T, Q_2 K_3^T, Q_2 K_4^T, \dots, Q_2 K_6^T \quad (76)$$

Find all the scores over attention after softmaxing you'll get top related words. Now vary i and j for all $e \in X, Q, K$.

E. Step 5 & 6: Scaling and Softmax

Step 5 (Scaling): The dot product is scaled to prevent vanishing gradients:

$$\frac{Q_i K_j^T}{\sqrt{d_k}} \quad (77)$$

Step 6 (Softmax): Convert scores to probabilities:

$$\alpha_{ij} = \frac{e^{\text{score}_{ij}}}{\sum_j e^{\text{score}_{ij}}} \quad (78)$$

Where $0 \leq \alpha_{ij} \leq 1$ and $\sum_j \alpha_{ij} = 1$. The final output Z_i is computed as:

$$Z_i = \sum_j \alpha_{ij} V_j \quad (79)$$

Every token contains info from all other tokens.

F. Step 7: Multi-Head Attention

Instead of one attention mechanism, $h_1, h_2, \dots, h_m$ multiple attention heads with separate W_Q, W_K, W_V matrices are used.

$$\text{Multihead} = \text{Concat}(\text{head}_1, \dots, \text{head}_m)W_O \quad (80)$$

G. Step 8: Add and Norm & Feed Forward Network

Add and Norm: Let $Y = \text{Attention}(X)$. The residual connection and normalization is applied as:

$$Y_1 = \text{LayerNorm}(X + Y) \quad (81)$$

Here, X is the input and Y is the learned transformation. This enables deeper networks by preventing vanishing gradients.

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (82)$$

This ensures activation stays stable.

Feed Forward Network: A small neural network applied to every token:

$$FFN(x) = W_2(\text{ReLU}(W_1x + b_1)) + b_2 \quad (83)$$

Suppose $W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ and $x = [2, 3]$, with $W_2 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$. Then $W_1x = [2, 3]$ and $W_2x = [4, 6]$. This allows non-linear feature extraction.

H. Step 9 & 10: Encoder Output

Step 9 (Add and normalize again):

$$Y_2 = \text{LayerNorm}(Y_1 + FFN(Y_1)) \quad (84)$$

This completes one encoder layer. The output is passed into the next encoder layer. After many layers, the sequence transforms from word embedding $\rightarrow$ attention layers $\rightarrow$ contextual embeddings.

Step 10 (Encoder output): Eventually, $H = [h_1, h_2, \dots, h_n]$. These vectors contain contextual information.

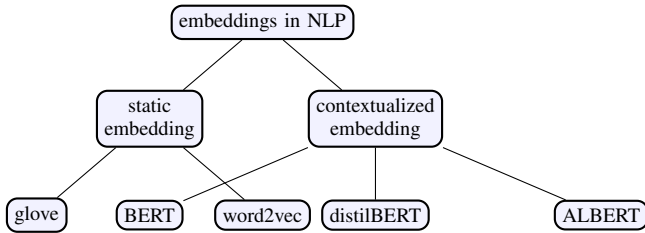


Fig. 14. Classification of static and contextualized NLP embeddings.

I. Step 11: Decoder Input

The decoder receives:

- Encoder output H
- Previously generated tokens

J. Step 12: Masked Multi-Head Attention

This is the same as self-attention except future tokens are hidden. A mask matrix M is added, so that future words can't be seen from encoder output.

$$\text{attention} = \text{SoftMax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) \quad (85)$$

K. Step 13: Encoder - Decoder Attention

In this layer:

$$Q = \text{Decoder}$$

$$K, V = \text{Encoder}$$

This allows the decoder to focus on relevant encoder information.

L. Step 14: Feed Forward + Add & Normalize + Linear Layer

The output is passed through a feed-forward layer:

$$FFN(x) = W_2(\text{RELU}(W_1x + b_1)) \quad (86)$$

For the linear layer, the decoder output $\rightarrow h$. Then $z = Wh + b$. The output dimension is $|\text{vocab}|$. If there are 50000 words, then $z \in \mathbb{R}^{50000}$. Each element is a score for one possible word.

M. Step 15: Softmax

Convert score to probabilities. The highest probability is chosen.

$$P(\text{word}_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (87)$$

XIX. SEMANTIC COMMUNICATION ARCHITECTURE

The Transformer encoder-decoder architecture seamlessly maps sequence intent. Input tokens $\rightarrow$ encoder $\rightarrow$ contextual semantic representation $\rightarrow$ decoder $\rightarrow$ output tokens.

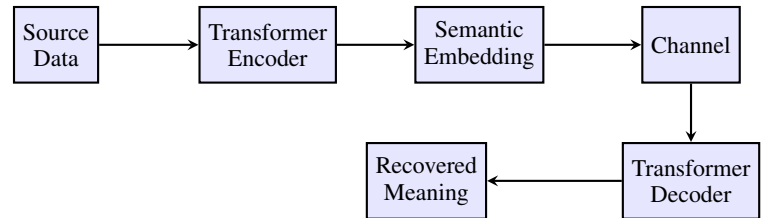


Fig. 15. Semantic communication data flow mapping meaning directly over a channel.

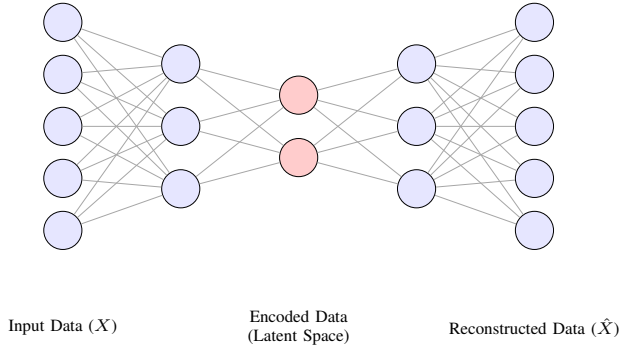


Fig. 16. Deep JSCC Autoencoder Architecture compressing and reconstructing data over a channel.

XX. JOINT SOURCE CHANNEL CODING (JSCC)

A. Limitations of Traditional Systems

In traditional communication systems, source coding and channel coding are designed independently based on the separation principle, provided that the source coding rate is less than the channel capacity ($R_s < C$). However, Shannon's theorem fundamentally assumes infinite block lengths, unlimited computational resources, and perfect encoding.

Modern applications such as autonomous vehicles, Internet of Things (IoT) devices, and AR/VR systems violate these theoretical assumptions. To address this, Joint Source Channel Coding (JSCC) ensures ultra-low latency, facilitates short packet transmission, and maximizes energy efficiency by combining source and channel coding into a single optimization framework.

B. End-to-End Deep Learning Architecture

Deep learning-based JSCC models the entire communication chain as a single, communication-aware autoencoder. This is an unsupervised neural network that learns to compress data into a lower-dimensional latent space and reconstruct it back to its original form.

The mathematical pipeline of the autoencoder is defined as follows:

- 1) **Encoder Neural Network:** Maps the source data into channel symbols:

$$Z = f_\theta(X) \quad (88)$$

- 2) **Channel Layer:** Introduces physical distortion and noise:

$$Y = H(Z) \quad (89)$$

- 3) **Decoder Neural Network:** Reconstructs the data from the noisy received signal:

$$\hat{X} = g_\phi(Y) \quad (90)$$

The entire communication pipeline can therefore be expressed as a single nested function:

$$\hat{X} = g_\phi(H(f_\theta(X))) \quad (91)$$

C. Channel Modeling as a Neural Layer

To enable end-to-end learning, the physical communication channel is inserted directly into the neural network during training as a non-trainable parameter layer.

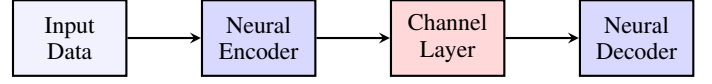


Fig. 17. Block Diagram of the Deep Learning Based JSCC Pipeline.

Two primary channel models are integrated into the network:

1) **AWGN Channel Layer:** The Additive White Gaussian Noise (AWGN) layer models thermal noise at the receiver. The decoder learns to mathematically compensate for this distribution:

$$Y = X + N \quad \text{where } N \sim \mathcal{N}(0, \sigma^2) \quad (92)$$

2) **Rayleigh Fading Layer:** To simulate multi-path propagation environments, a Rayleigh fading coefficient (h) is introduced:

$$Y = hX + N \quad (93)$$

The neural network experiences fading during the training process, thereby autonomously learning robust transmission strategies without explicit pilot signals.

D. Power Constraint Normalization

To ensure the transmitter emits finite energy over the physical channel, the encoder outputs (Z) are explicitly normalized. For an available transmit power P and sequence length k , the constraint is bounded by:

$$\frac{1}{k} \sum_{i=1}^k z_i^2 \leq P \quad (94)$$

This is enforced dynamically within the network layers via the following transformation:

$$Z = \sqrt{kP} \frac{Z}{\|Z\|} \quad (95)$$

E. Optimization and Backpropagation

A fundamental advantage of Deep JSCC is the presence of a single loss function and a unified optimization process. The global loss function propagates backward from the receiver to the transmitter through the differentiable channel layer.

$$\text{Goal: } \min_{\theta, \phi} L(X, \hat{X}) \quad (96)$$

During backpropagation:

- $\frac{\partial L}{\partial \phi}$ updates the receiver (decoder) parameters.
- $\frac{\partial L}{\partial \theta}$ updates the transmitter (encoder) parameters.

The loss configuration depends on the datatype being transmitted:

- **For Images:** Mean Squared Error (MSE) is utilized.

$$L = \frac{1}{N} \sum_{i=1}^N (X_i - \hat{X}_i)^2 \quad (97)$$

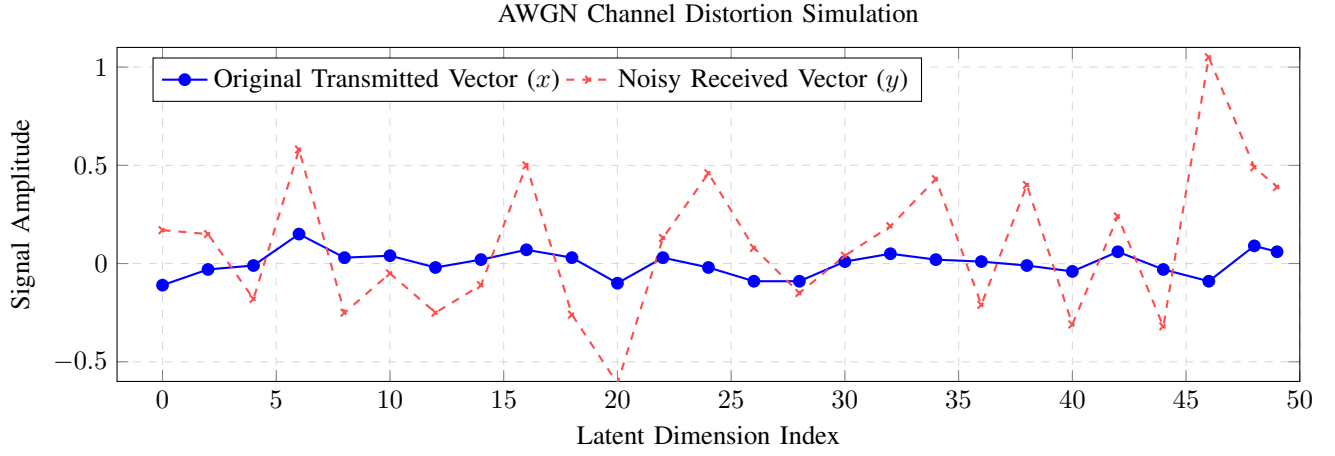


Fig. 18. Visualization of original latent embeddings versus heavily distorted signals passing through an AWGN layer.

- **For Text:** Cross-Entropy Loss evaluates character/token probabilities.

$$L = - \sum y_i \log(\hat{y}_i) \quad (98)$$

F. Quality vs. Channel Degradation (The Cliff Effect)

Traditional communication protocols suffer from the "Cliff Effect". If the SNR falls below a specific threshold (T), the forward error correction fails completely, resulting in a sudden collapse of communication quality.

Deep JSCC naturally avoids this. Since the quality is a direct learned function of SNR ($Quality = f(SNR)$), signal degradation is graceful and smooth, making it vastly superior for volatile wireless environments.

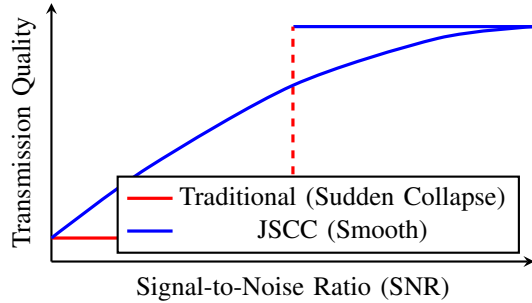


Fig. 19. Comparison of signal recovery quality: The traditional Cliff Effect versus the graceful degradation of JSCC.

G. JSCC for Semantic Communication

When extended into semantic communication, the Deep JSCC architecture is enveloped by Large Language Models (LLMs) or Transformer blocks to process intent rather than structural data.

The loss function transitions from pixel/bit accuracy to Semantic Cosine Similarity, measuring the multi-dimensional angle between the original thought embedding (E_X) and the recovered meaning embedding ($E_{\hat{X}}$):

$$L_{sem} = 1 - \frac{E_X \cdot E_{\hat{X}}}{\|E_X\| \|E_{\hat{X}}\|} \quad (99)$$

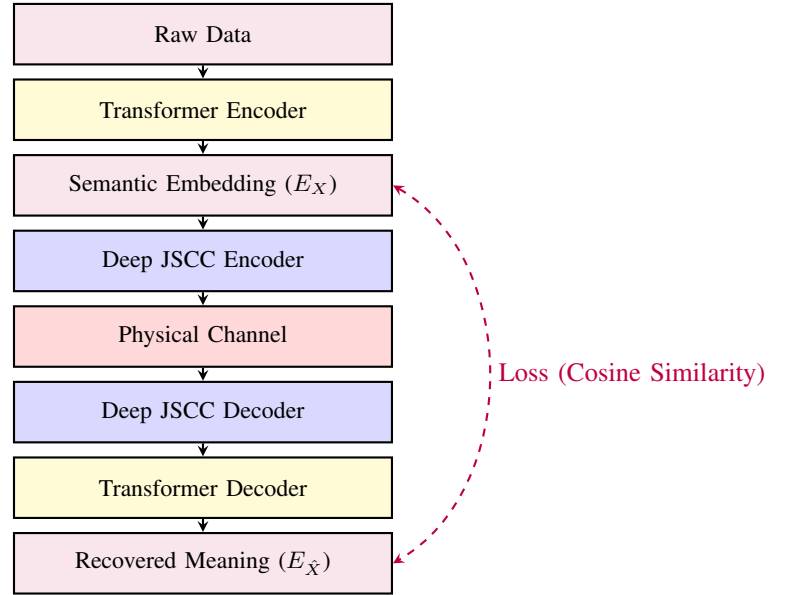


Fig. 20. End-to-End Pipeline of JSCC optimized for Semantic Communication.

XXI. SYNTACTIC DROP AND INFORMATION BOTTLENECK

A. Information Bottleneck Theory

The Information Bottleneck theory states that as a sentence gets longer, the fixed-size embedding vector doesn't have enough space to store every syntactic detail. Consequently, it drops the syntactic structure and preserves the semantic structure.

B. True Bottleneck Models vs. Matrix Output Models

In a true bottleneck model—which extracts only the CLS token or uses a fixed-size seq2seq bottleneck—the opposite behavior occurs. Both a small sentence and a large sentence would be compressed to the same n -dimensional vector. In this case, a long sentence would collapse much faster because

its mathematical space is more crowded and highly sensitive to noise.

Conversely, a long sentence actually survives better than a short sentence in models like BART, which outputs a matrix of vectors (one vector for every single word). If noise destroys one word's vector, BART uses the other 19 words to guess the missing word. The short sentence simply didn't have enough surrounding context.

XXII. SEQUENCE-TO-SEQUENCE (SEQ2SEQ) MODELS

Architectures such as T5 (Text-to-Text Transfer Transformer) and BART (Bidirectional and Auto-Regressive Transformer) are seq-seq models highly suitable for semantic extraction, semantic reconstruction, text compression, translation, etc. For a given input $X = (x_1, x_2, \dots, x_n)$, the model learns the distribution $P(Y|X)$.

A. T5 Architecture and Tokenization

T5 uses a full transformer encoder-decoder configuration. The T5 Tokenizer utilizes SentencePiece tokenization, which breaks words into subwords. For a text string T , the sequence is represented as:

$$f(T) = [t_1, t_2, \dots, t_n] \quad (100)$$

Each token t_i is mapped to an integer ID, from which an embedding vector is derived:

$$e_i = \text{Embedding}(t_i), \quad e_i \in \mathbb{R}^d \quad (101)$$

Each encoder layer computes the sequence iteratively:

$$H^{(l)} = \text{FFN}(\text{attention}(H^{(l-1)})) \quad (102)$$

After processing through multiple layers, the final state array $H = [h_1, h_2, \dots, h_n]$ contains the consolidated semantic information.

B. T5 Decoder and Span Corruption

The T5 decoder generates the output autoregressively at step t :

$$P(y_t | y_{<t}, H) \quad (103)$$

Meaning, the current token strictly depends on:

- Previously generated tokens ($y_{<t}$)
- Encoder semantic information (H)

The overall sequence generation probability follows:

$$P(Y|X) = \prod_{t=1}^m P(y_t | y_{<t}, X) \quad (104)$$

Instead of predicting the next word natively, T5 performs **Span Corruption**: the model learns from corrupted text to deduce missing text. This mechanism forcefully instills deep semantic understanding.

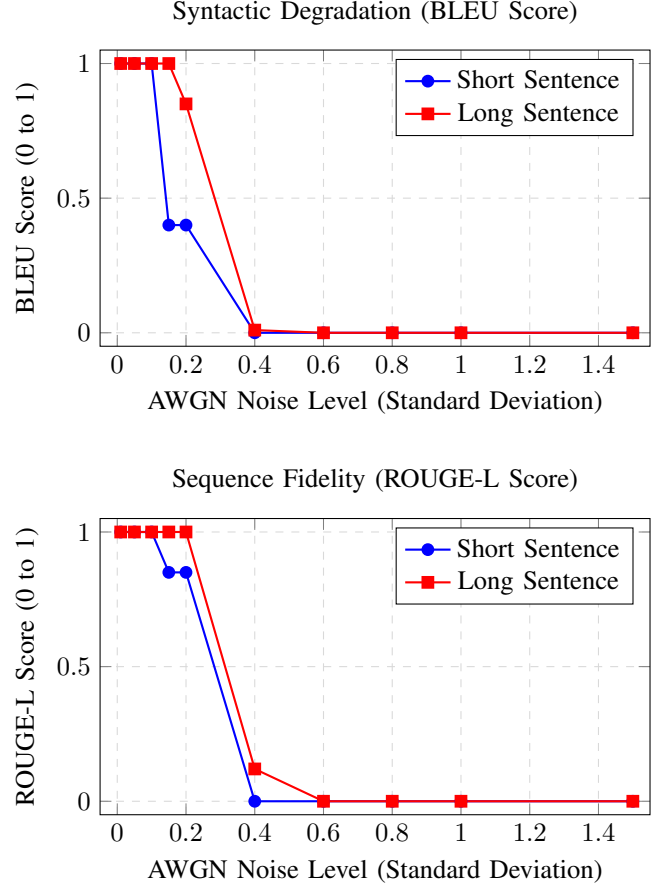


Fig. 21. Impact of AWGN Noise Level on Syntactic Degradation and Sequence Fidelity for short vs. long sentences.

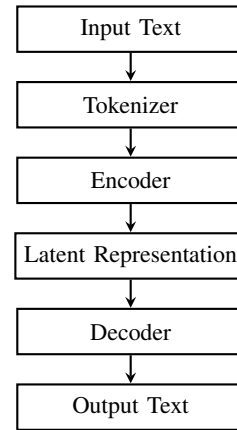


Fig. 22. Structural flow of the T5 Architecture.

XXIII. BART ARCHITECTURE AND ROLE IN COMMUNICATION

To contextualize the evolutionary mapping of transformer models:

- **BERT:** Encoder only
- **GPT:** Decoder only
- **BART:** Encoder + Decoder

A. BART Corruption Process

The fundamental BART architecture routes from the input $[i/p]$ into a $[transformer\ encoder]$ block to extract a $[latent\ representation]$, which is then parsed by the $[transformer\ decoder]$ to yield the $[o/p]$. The defining noise-corruption and training pipeline is:

- Original data: X
- Noise function: $\tilde{X} = N(X)$
- Encoder processes: $\tilde{X}$
- Decoder reconstructs: $\hat{X}$
- Training objective: $\min \rightarrow \log_P(X | \tilde{X})$

B. Role in Semantic Communication

In a functional physical layer, these NLP architectures operate as Joint Source-Channel Coding (JSCC) entities.

XXIV. TOKENIZATION THEORY

Before compressing a sentence into a mathematical vector, we must convert the raw text into discrete integers that the neural network can process. This foundational process is called **Tokenization**.

Neural networks cannot read strings directly. Therefore, we must scan the dataset, build a comprehensive lookup table, and map every unique word to a unique integer representation.

TABLE IX
EXAMPLE OF A TOKENIZATION LOOKUP TABLE

Word (String)		Integer ID
Arpit	→	4312
School	→	1284
Rescue	→	1001
VLSI	→	724
Aadya	→	9901
AI	→	1234

To train a robust model over a noisy communication channel, we must define 4 specialized tokens to handle structural variations and sequence control:

A. 1. <PAD> (ID: 0)

If sentence A has 5 words and sentence B has 10 words, we must append 5 <PAD> tokens to sentence A so that both sequences possess exactly 10 words. This standardisation is strictly necessary as neural networks process data in parallel batches and require uniform matrix dimensions.

B. 2. <UNK> (ID: 1)

When encountering a word that has never been seen before in the training dataset, the model replaces it with the <UNK> (Unknown) token. This forces the Long Short-Term Memory (LSTM) network or Transformer to rely entirely on the surrounding context to guess the meaning of the unknown word.

C. 3. <BOS> (ID: 2)

When an LSTM decoder wants to reconstruct a sentence from a noisy semantic vector, it needs a definitive trigger to know it is time to generate the very first word. To initiate this generation sequence, we feed it the <BOS> (Beginning of Sentence) token.

D. 4. <EOS> (ID: 3)

If the <EOS> (End of Sentence) token is not present, the decoder will theoretically keep predicting words forever. Once the <EOS> token is predicted, the receiver network definitively knows the sequence transmission is complete.

XXV. TOKENIZATION & MATHEMATICS

A. Zipf's Law & The <UNK> Danger Zone

In linguistics and information theory, the semantic information carried by a word is inversely proportional to its frequency of occurrence. Mathematically, the information content $I(x)$ of a symbol x is defined as:

$$I(x) = -\log P(x) \quad (105)$$

Words like "the", "a", or "is" carry almost no significant semantic meaning, but they occur a massive number of times within a dataset. Conversely, highly specific words occur rarely but carry the bulk of the sentence's meaning.

When creating the vocabulary lookup table, setting a minimum frequency threshold filters out rare words (replacing them with <UNK>) to save memory. However, if you keep the minimum frequency threshold too high, the majority of the true data volume and semantic meaning is destroyed.

B. The Padding Tax

While padding is necessary for matrix operations, transmitting a <PAD> token through a physical 6G channel consumes actual electrical power and physical bandwidth for absolutely zero semantic gain. This phenomenon is referred to as the **padding tax**.

Because the neural network input matrix must be sized to accommodate the absolute longest sentence in the dataset (e.g., 503 words), every shorter sentence (which averages around 80.9 words) must be artificially padded up to length 503, drastically inflating transmission overhead.

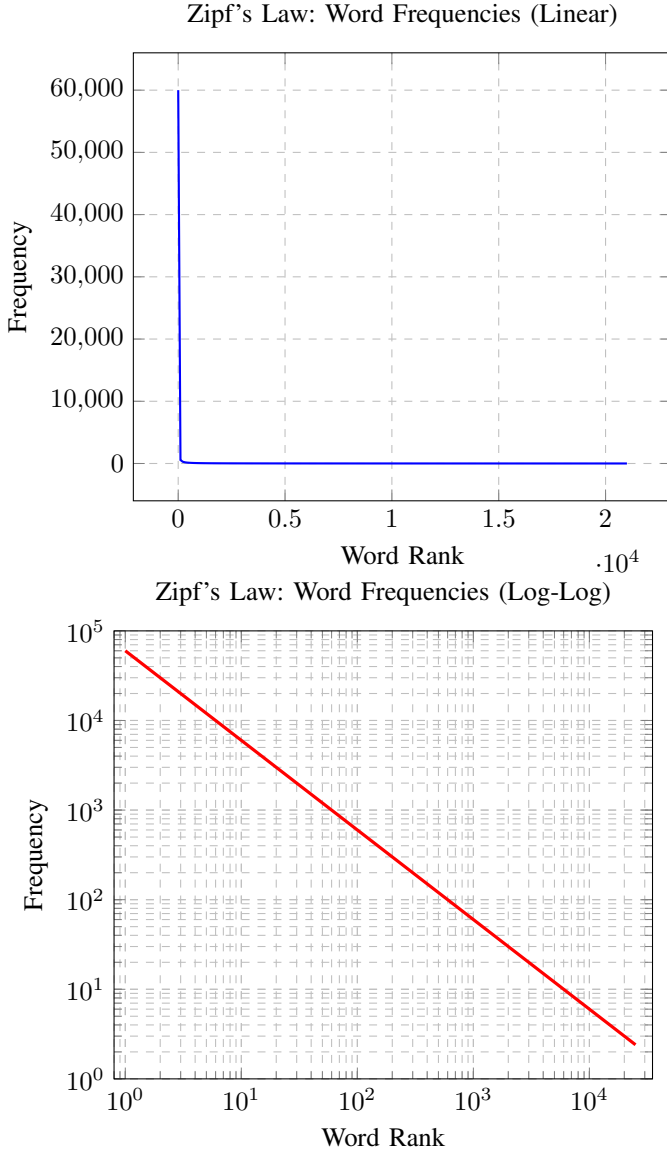


Fig. 23. Visual demonstration of Zipf's Law showing the inverse relationship between word rank and frequency across linear and log-log scales.

C. Shannon Entropy

Derived directly from the token probabilities, the Shannon Entropy establishes the theoretical lower bound of compression. It defines the absolute minimum average number of bits required to transmit a symbol:

$$H = - \sum P(x) \log_2 P(x) \quad (106)$$

In the context of semantic networks, entropy guides how densely we can pack our numerical vectors without violating the fundamental limits of information reconstruction at the receiver end.

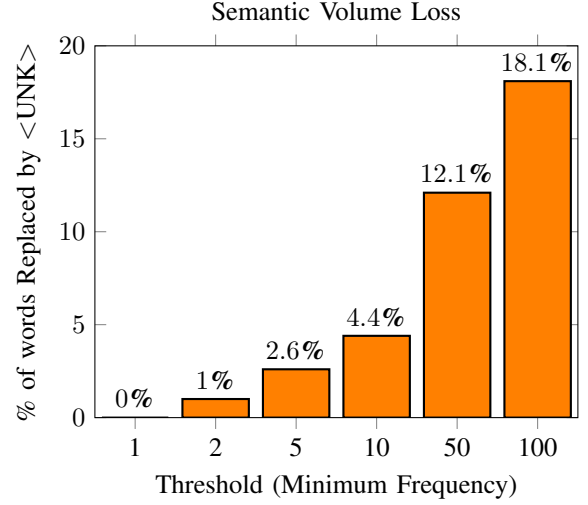


Fig. 24. Dependence of semantic volume loss on the minimum frequency threshold. A higher threshold heavily degrades the underlying meaning of the dataset.

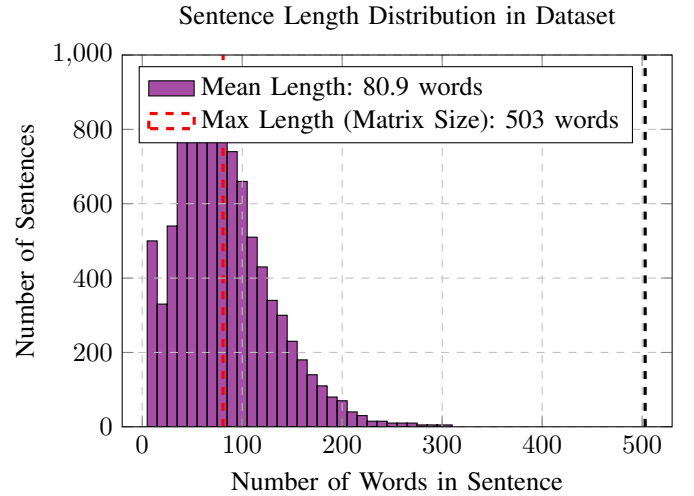


Fig. 25. Visualization of the padding tax. Because the matrix must accommodate the maximum length sequence (503 words), the vast majority of shorter sentences (mean 80.9) are heavily padded with useless zeros.

June 20, 2026

XXVI Implementation of AWGN channel

XXVI.1 Physical Environment Simulation

In a real-world physical environment, vectors do not pass between transmission layers perfectly due to inherent signal degradation and interference. To accurately simulate these physical channel conditions within our deep learning architecture, we insert a specialized neural layer in the middle of our network: the AWGN channel.

XXVI.2 Additive Gaussian (AWGN) White Noise

The AWGN channel is mathematically defined by three fundamental properties:

- **A → Additive:** The noise is directly added to the signal, rather than being multiplicative.
- **W → White:** The noise possesses equal power across the entire frequency spectrum.
- **G → Gaussian:** The magnitude of the noise follows a normal (bell curve) representation over time.

XXVI.3 SNR Mathematics and Noise Power

The ratio of signal power to noise power is defined as the Signal-to-Noise Ratio (SNR).

$$SNR = \frac{P_{signal}}{P_{noise}} \quad (107)$$

From this relationship, the required noise power can be derived as:

$$P_{noise} = \frac{P_{signal}}{SNR} \quad (108)$$

The signal power (P_{signal}) is calculated as the expected value of the squared magnitude of the transmitted vector X :

$$P_{signal} = \mathbb{E}[|X|^2] \quad (109)$$

Because SNR is typically provided in decibels (dB), we must convert it to a linear scale before applying it to the tensor:

$$SNR_{linear} = 10^{\left(\frac{SNR_{dB}}{10}\right)} \quad (110)$$

XXVI.4 PyTorch Implementation

In PyTorch, the function `torch.randn` generates a random tensor drawn from a standard normal distribution, $\mathcal{N}(0, 1)$. To apply the precise amount of noise corresponding to the target SNR, we scale this base distribution by the standard deviation, which is the square root of the variance (noise power).

The algorithmic implementation follows these steps:

1. Calculate Noise Power:

$$noise_power = \frac{signal_power}{snr_linear} \quad (111)$$

2. Calculate Standard Deviation:

$$noise_std_dev = \text{torch.sqrt}(noise_power) \quad (112)$$

3. Generate Scaled Noise:

$$noise = \text{torch.randn_like}(X) \times noise_std_dev \quad (113)$$

4. Calculate Received Signal:

$$received = X + noise \quad (114)$$

- Batch Size = 32 (representing 32 sentences)

- Sequence Length = 10 (representing 10 words per sentence)

- Embedding Dimension = 64

Before passing through the embedding layer, the tensor shape is (32, 10). After the embedding layer, the shape becomes (32, 10, 64).

As the tensor processes through the LSTM, the hidden dimension (e.g., 128) maps the current timestep embedding $e_t \in \mathbb{R}^{64}$ into a context space. The network output shape after the LSTM layer becomes (1, 32, 128), successfully converting the input to a context space of 128 dimensions.

XXVI.5 Visualizing Channel Noise Effects

To comprehensively understand the destructive capabilities of AWGN, we observe its effects across varying SNR levels on both a continuous analog waveform and a discrete vector point in a 2D space.

XXVII The LSTM Model

XXVII.1 Embedding Dimension vs. Hidden Dimension

In the context of the Long Short-Term Memory (LSTM) model, data representations are fundamentally governed by two dimensional spaces:

- **Embedding Dimension:** Controls how words are mathematically represented before transmission.
- **Hidden Dimension:** Controls the size of the semantic message that is transmitted through the channel.

For example, if the embedding dimension is set to 64, all words in the sequence are converted into a 64-dimensional vector representation. The embedding matrix size is given by:

$$\text{Embedding Matrix} = (\text{Vocab Size} \times \text{Embed Dim}) \quad (115)$$

Consider a data transmission scenario where:

XXVII.2 LSTM Architecture and Memory States

The LSTM receives two crucial pieces of information at any timestep t :

1. e_t : The current meaning (from the embedding layer).
2. h_{t-1} : The previous sentence context.

The internal architecture relies on two distinct memory pathways. The **Cell State** (c_t) acts as the long-term memory, running through the cell with minimum alteration. Conversely, the **Hidden State** (h_t) acts as the short-term working memory, capturing immediate context, continuously updating, and serving as the direct output of the LSTM cell.

XXVII.3 Gate Computations and Dimensionality

The transition of features from the 64-dimensional embedding to the 128-dimensional hidden space is executed via matrix multiplications within the four neural gates.

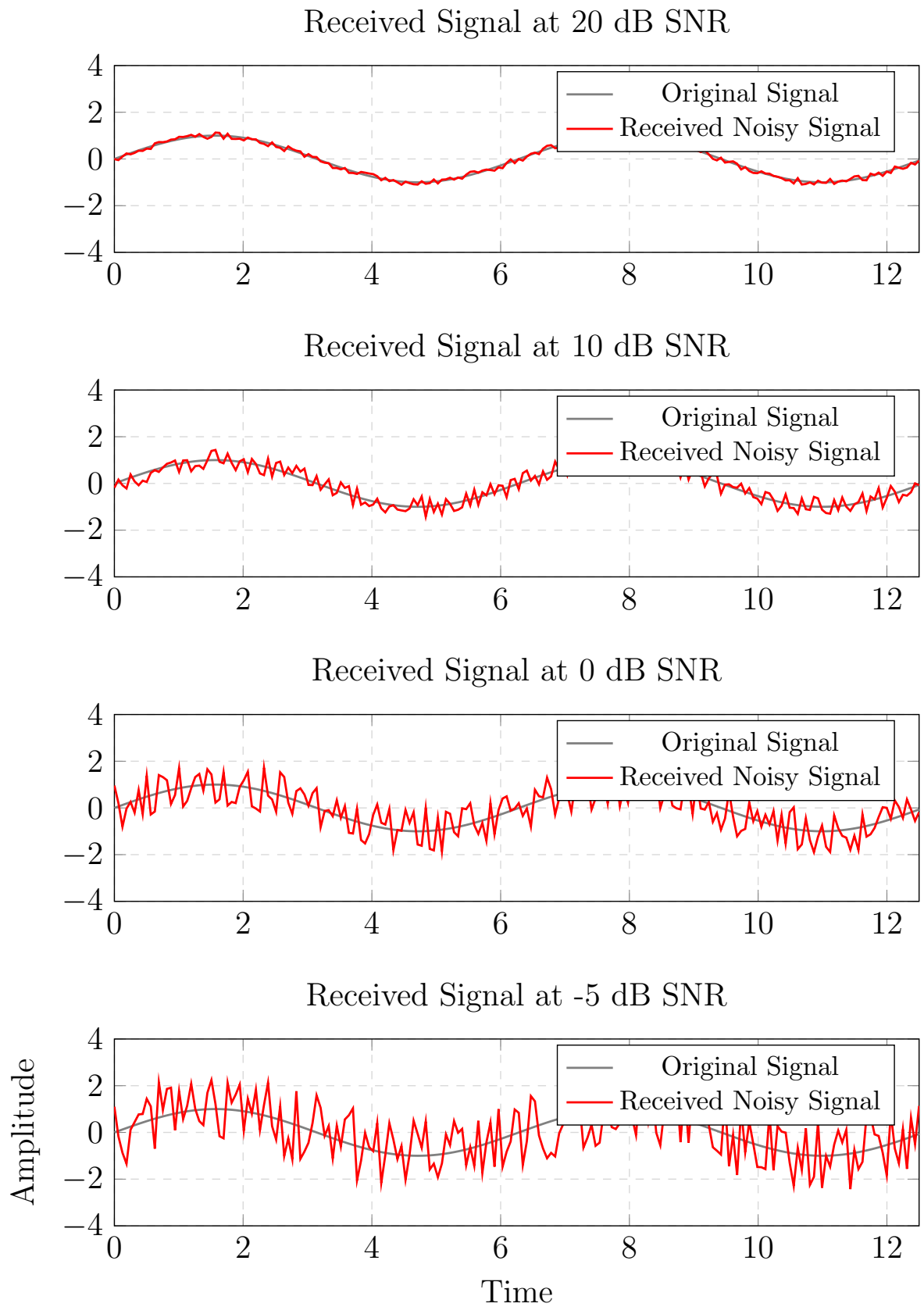


Figure 26: Effect of Additive White Gaussian Noise (AWGN) on a continuous sine wave under degrading Signal-to-Noise Ratio (SNR) conditions.

Vector Space at 20 dB SNR Vector Space at 10 dB SNR Vector Space at 0 dB SNR

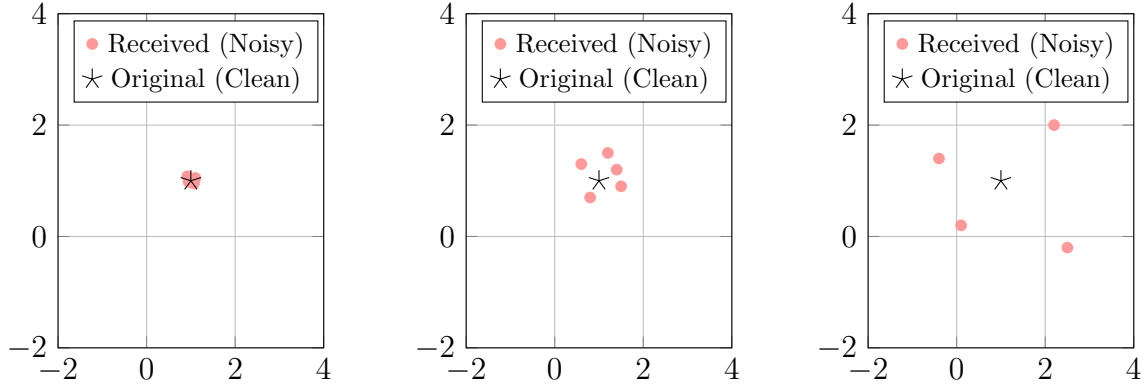


Figure 27: Scattering effect of AWGN on a 2D transmitted vector point (1,1) across decreasing SNR levels.

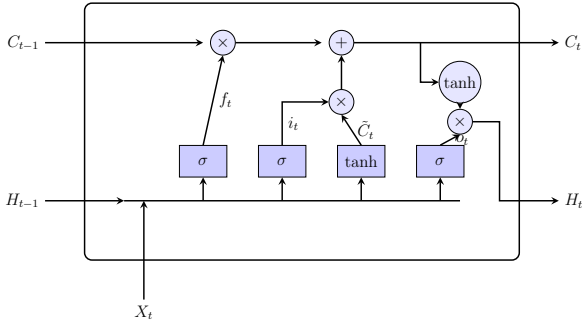


Figure 28: Internal computational diagram of an LSTM block detailing gate operations and memory state pathways.

1. Forget Gate: Determines what information to discard from the cell state.

$$f_t = \sigma(W_f e_t + U_f h_{t-1} + b_f) \quad (116)$$

$$W_f \in \mathbb{R}^{128 \times 64}, \quad e_t \in \mathbb{R}^{64}$$

$$\implies W_f e_t \in \mathbb{R}^{128}$$

$$U_f \in \mathbb{R}^{128 \times 128}, \quad h_{t-1} \in \mathbb{R}^{128}$$

$$\implies U_f h_{t-1} \in \mathbb{R}^{128}$$

Due to matrix multiplication $([128 \times 64] \times [64 \times 1] \rightarrow [128 \times 1])$, the 64-dimension embedding is converted to 128 dimensions, establishing $f_t \in \mathbb{R}^{128}$.

2. Input Gate and Candidate Memory: Determines what new information is stored in the cell state.

$$i_t = \sigma(W_i e_t + U_i h_{t-1} + b_i) \quad (117)$$

$$\tilde{C}_t = \tanh(W_c e_t + U_c h_{t-1} + b_c) \quad (118)$$

Following identical dimensional logic, $W_i, W_c \in \mathbb{R}^{128 \times 64}$ and $U_i, U_c \in \mathbb{R}^{128 \times 128}$, yielding $i_t, \tilde{C}_t \in \mathbb{R}^{128}$.

3. Output Gate: Determines the output based on the filtered cell state.

$$o_t = \sigma(W_o e_t + U_o h_{t-1} + b_o) \quad (119)$$

$$o_t \in \mathbb{R}^{128}$$

4. Memory State Creation: The long-term and short-term memory states are updated through element-wise operations ($\odot$):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (120)$$

$$h_t = o_t \odot \tanh(C_t) \quad (121)$$

XXVII.4 Internal Computation and Parameter Count

While conceptually discrete, computational frameworks like PyTorch optimize this by concatenating the inputs at every timestep. The LSTM actually computes $[e_t; h_{t-1}]$.

$$\text{Concatenation} \rightarrow 64 + 128 = 192 \text{ dims} \quad (122)$$

The framework internally evaluates a combined weight matrix $W[e_t; h_{t-1}] + b$, where $W \in \mathbb{R}^{128 \times 192}$, effectively mapping $192 \rightarrow 128$.

The total number of parameters required to define the network can be analyzed. For a single gate, the parameter breakdown is:

$$(128 \times 64) + (128 \times 128) + (128) = 24,704 \quad (123)$$

This formulation represents:

- 128×64 : Evaluates how each of the 64 embedding features affects each of the 128 memory units.
- 128×128 : Evaluates how the previous memory influence maps to the current memory.
- **128**: The bias vector (b).

Since the standard LSTM cell contains exactly 4 independent gates, the total trainable parameters evaluate to:

$$4 \times 24,704 = 98,816 \text{ parameters} \quad (124)$$

XXVIII LSTM Architecture (JCC Modelling)

XXVIII.1 End-to-End System Model

The Joint Source-Channel Coding (JCC) model fundamentally operates on an end-to-end framework parameterized by deep learning structures. The overarching mathematical representation of the received and decoded signal $\hat{X}$ is formulated as:

$$\hat{X} = g_\phi(C(f_\theta(X))) \quad (125)$$

where:

- f_θ represents the **semantic encoder**.
- C represents the **AWGN channel**.
- g_ϕ represents the **semantic decoder**.

XXVIII.2 Semantic Encoder

The initial stage of the transmission pipeline is the semantic encoder. It maps discrete token IDs into a continuous dense vector space. The embedding vector e_t for a given token sequence x_t is extracted via the embedding function E :

$$e_t = E(x_t) \quad (126)$$

The embedding matrix E belongs to the real coordinate space defined by the vocabulary size V and the embedding dimension d_e :

$$E \in \mathbb{R}^{V \times d_e} \quad (127)$$

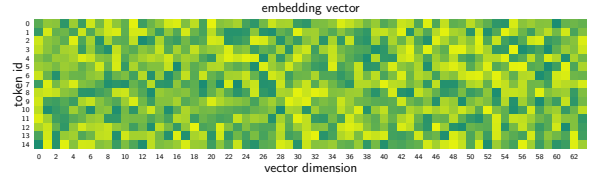


Figure 29: Heatmap visualization of the dense embedding vector space mapped from discrete token IDs, where $V = 15$ and $d_e = 64$.

XXVIII.3 LSTM Semantic Compression

Following the embedding stage, the sequence of embeddings is processed iteratively by a Long Short-Term Memory (LSTM) network to achieve semantic compression:

$$(hidden, cell) = \text{self.lstm}(embedded) \quad (128)$$

After processing all the words in the sequence, the terminal hidden state h_T contains the concentrated semantic information from the sentence. The encoder returns the tuple (h_T, C_T) to form the latent message.

XXVIII.4 Channel Input

The latent message components are concatenated to form the final transmitted signal vector z :

$$z = [h_T, C_T] \quad (129)$$

The dimensionality of this bottleneck vector d_z is exactly twice the hidden dimension H . Given $H = 128$:

$$d_z = 2H = 2 \times 128 = 256 \quad (130)$$

This composite vector acts as the transmitted signal over the physical channel.

XXVIII.5 Noise Modelling

During physical transmission, Additive White Gaussian Noise (AWGN) corrupts the latent message. The received noisy representations of the hidden and cell states are:

$$\tilde{h} = h + n_h \quad (131)$$

$$\tilde{c} = c + n_c \quad (132)$$

The total signal power P_s is calculated as the mean squared amplitude across N elements:

$$P_s = \frac{1}{N} \sum_{i=1}^N x_i^2 \quad (133)$$

Using the defined Signal-to-Noise Ratio in decibels (SNR_{dB}):

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_s}{P_n} \right) \quad (134)$$

The required noise power P_n (which equates to the variance σ^2 of the Gaussian noise) is derived as:

$$P_n = \frac{P_s}{10^{SNR_{dB}/10}} \quad , \quad \sigma^2 = P_n \quad (135)$$

XXVIII.6 Semantic Decoder

At the receiver side, the semantic decoder reconstructs the intent. During training, the true previous word is fed to the decoder

(Teacher Forcing) to calculate the conditional probability:

$$P(x_t | x_{t-1}, h_t) \quad (136)$$

The decoder outputs a hidden state h_t , which is converted into vocabulary probabilities via a linear transformation:

$$o_t = Wh_t + b \quad \left(o_t \in \mathbb{R}^V \right) \quad (137)$$

This output o_t is pushed through a Softmax function to generate a valid probability distribution over the vocabulary:

$$P(w_i) = \frac{e^{o_i}}{\sum_j e^{o_j}} \quad (138)$$

The predicted token $\hat{x}_t$ is then selected via an argmax operation:

$$\hat{x}_t = \arg \max P(w_i) \quad (139)$$

XXVIII.7 Training Objective

The overarching training objective of the system is to minimize the Cross-Entropy Loss L over the sequence of length T :

$$L = - \sum_{t=1}^T \log P(x_t) \quad (140)$$

Fundamentally, the encoder and decoder architectures jointly learn to minimize this discrepancy between the original and reconstructed messages:

$$\min \mathcal{L}(g_\phi(C(f_\theta(x))), x) \quad (141)$$

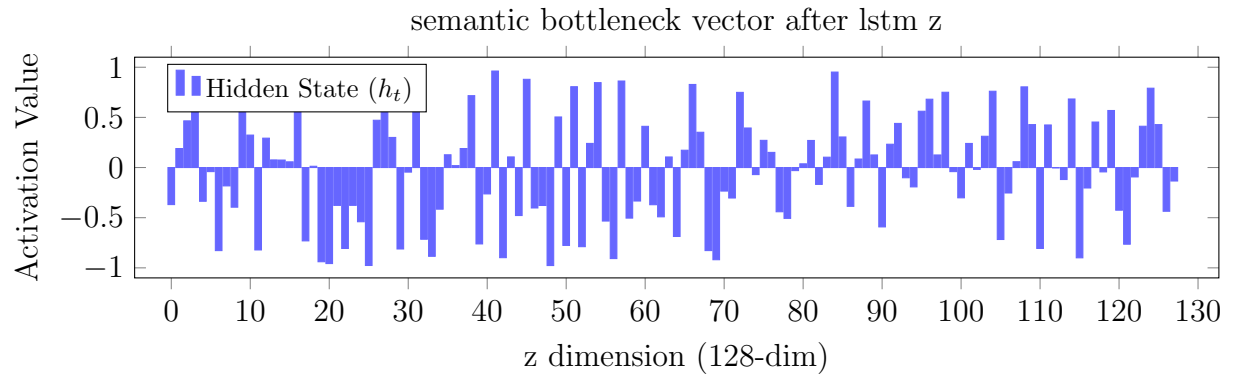


Figure 30: Activation values of the semantic bottleneck vector representing the mapped LSTM hidden state prior to channel transmission.

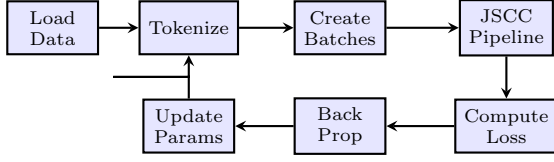


Figure 32: Iterative training pipeline of the Semantic Communication Model.

30 Training Phase

Now that the semantic communication model has been successfully implemented, the next step is to train the network so that it can learn a robust end-to-end mapping between the transmitted sentence and its reconstructed version at the receiver. During training, all trainable parameters are jointly optimized through backpropagation, allowing the encoder, channel representation, and decoder to function as a unified Joint Source-Channel Coding (JSCC) system.

30.1 Pipeline Overview

The training process operates iteratively over the dataset. Each iteration begins with loading raw textual data and concludes with parameter updates obtained through gradient-based optimization. The overall workflow is illustrated in Figure 32.

30.2 Optimization Problem

Training aims to determine the optimal parameter set that minimizes the reconstruction loss over the entire dataset. Formally,

$$\min_{\theta} \mathbb{E}_{x \sim \mathcal{D}} [L(x, \hat{x})] \quad (143)$$

where

$$\hat{x} = \text{Decoder}(\text{Channel}(\text{Encoder}(x))) \quad (144)$$

denotes the reconstructed sentence generated by the receiver.

The trainable parameter set is

$$\theta = \{W_e, \Theta_{enc}, \Theta_{dec}, W_{fc}, b_{fc}\} \quad (145)$$

where

- W_e denotes the embedding matrix.
- Θ_{enc} denotes Encoder LSTM parameters.
- Θ_{dec} denotes Decoder LSTM parameters.
- W_{fc} and b_{fc} denote output layer weights and biases.

Furthermore,

- Batch Size (B): Number of samples processed simultaneously.

- Sequence Length (T): Number of tokens in a sentence.
- Vocabulary Size (V): Number of unique words in the tokenizer vocabulary.

30.3 Semantic Compression and Dimensionality

Let the tokenized sentence be represented as

$$x = (x_1, x_2, \dots, x_T) \quad (146)$$

where

$$x_i \in \{1, 2, \dots, V\}.$$

For a batch size of 100 and sequence length of 30, the dimensional transformations are

- Input Tokens: (100, 30)
- Embedding Output: (100, 30, 128)
- Encoder Hidden State: (1, 100, 256)
- Encoder Cell State: (1, 100, 256)

The final encoder states are

$$h_T \in \mathbb{R}^{256}, \quad c_T \in \mathbb{R}^{256} \quad (147)$$

and are concatenated to form

$$z = [h_T; c_T] \in \mathbb{R}^{512}. \quad (148)$$

This bottleneck vector performs semantic compression by summarizing information contained across the entire sentence into a compact latent representation. The objective is to preserve semantic meaning while removing redundancy prior to channel transmission.

30.4 Decoder Mechanics and Cross-Entropy Loss

The compressed representation is transmitted through the AWGN channel and reconstructed by the decoder.

At decoding timestep t ,

$$P(y_t | y_{<t}, \tilde{h}_T, \tilde{c}_T) \quad (149)$$

defines the probability of predicting the next token. Here $\tilde{h}_T$ and $\tilde{c}_T$ represent the channel-corrupted hidden and cell states.

During training, teacher forcing is employed. The ground-truth token at timestep $t - 1$ is supplied to predict the token at timestep t , thereby improving convergence speed and training stability.

The decoder output tensor has dimensions

$$(100, 30, V).$$

Let $\hat{y}_{ik}$ denote the predicted probability of token k at position i , and y_{ik} denote the corresponding one-hot target label. The Cross-Entropy loss is

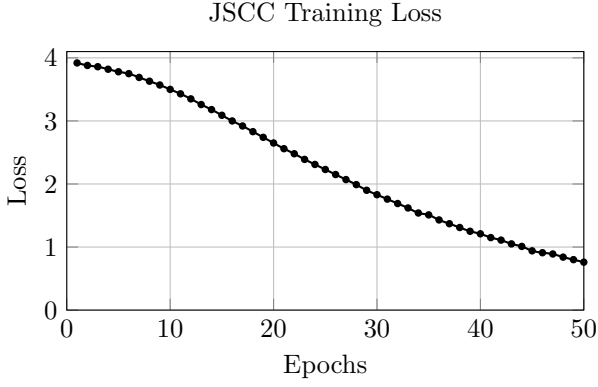


Figure 33: Progressive reduction of Cross-Entropy Loss across 50 training epochs.

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^V y_{ik} \log(\hat{y}_{ik}) \quad (150)$$

where PAD tokens are ignored during loss computation.

30.5 Backpropagation and Parameter Update

Once the loss has been computed, gradients are evaluated with respect to all trainable parameters:

$$\nabla_{\theta} L \quad (151)$$

The gradient flow propagates backward through the complete architecture:

Loss $\rightarrow$ FC Layer $\rightarrow$ Decoder LSTM $\rightarrow$ AWGN Channel

Although noise is injected by the AWGN channel, the operation remains differentiable with respect to trainable parameters, allowing end-to-end optimization of the complete JSCC system.

Parameter updates are performed using the Adam optimizer:

$$\theta \leftarrow \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (152)$$

where

- α is the learning rate.
- $\hat{m}_t$ is the bias-corrected first-moment estimate.
- $\hat{v}_t$ is the bias-corrected second-moment estimate.
- ϵ is a numerical stability constant.

31 Attention Mechanism and Noise Layer

31.1 Encoder Computations

The encoder computes the self-attention mechanism where each word decides which words matter within the sequence. The queries (Q), keys (K), and values (V) are generated linearly:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (153)$$

The attention scores are scaled to prevent vanishing gradients:

$$A = \frac{QK^T}{\sqrt{d_k}} \quad (154)$$

These scores are passed through a softmax function to generate probabilities, which are then multiplied by the value matrix:

$$P = \text{softmax}(A), \quad O = PV \quad (155)$$

31.2 Noise Layer

The physical channel introduces Additive White Gaussian Noise (AWGN) to the transmitted semantic vectors. The noise n follows a normal distribution:

$$n \sim \mathcal{N}(0, \sigma^2) \quad (156)$$

$$p(n) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-n^2/(2\sigma^2)} \quad (157)$$

The variance σ^2 is determined by the Signal-to-Noise Ratio (SNR):

$$\sigma^2 = \frac{P_{\text{signal}}}{SNR} \quad (158)$$

Given an SNR (dB) for training set to 5.0:

$$SNR = 10^{5/10} \approx 3.162 \quad (159)$$

$$\sigma^2 = \frac{1}{3.162} = 0.316 \implies \sigma = \sqrt{0.316} = 0.562 \quad (160)$$

Thus, the specific noise distribution during the training phase is $n \sim \mathcal{N}(0, 0.316)$.

31.3 Decoder Estimates

The decoder estimates the probability $P(y_t|y_{<t}, z_{\text{noisy}})$ for every token position. At each computational step, the decoder produces V (vocabulary size) probabilities.

$$p(y_t) \implies L = -\log p(y_t) \quad (161)$$

The total loss across all tokens in the sequence is:

$$L = \sum_t \log p(y_t) \quad (162)$$

For a batch of size B , the averaged batch loss is:

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B L_i \quad (163)$$

32 Optimization and Training Goal

32.1 Backpropagation

During backpropagation, the loss gradient $\nabla_{\theta} L$ is calculated. For every parameter θ_j in the network, the gradient $\frac{\partial L}{\partial \theta_j}$ is computed to determine the direction of optimization.

32.2 Adam Optimization

The Adam optimizer utilizes moving averages of the gradients to iteratively update the parameters. It computes the 1st and 2nd moments:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1\text{st moment}) \quad (164)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2\text{nd moment}) \quad (165)$$

The parameters are then updated using the bias-corrected moments, learning rate η , and a stability constant ϵ :

$$\theta = \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad \text{parameter update} \quad (166)$$

32.3 The Training Goal

The ultimate training objective is to optimize the parameter set θ over the expectation of the dataset (X, Y) by minimizing the negative log-likelihood (or maximizing the likelihood) over the noisy channel:

$$\min_{\theta} \mathbb{E}_{(X,Y)} [-\log P_{\theta}(Y|X, \text{AWGN})] \quad (167)$$

$$\max_{\theta} P_{\theta}(Y|X, \text{AWGN}) \quad (168)$$

33 T5 Model Upgrade

This architecture is significantly more advanced than LSTM-based semantic systems because it leverages a T5 model as both the semantic encoder and semantic decoder, while directly injecting channel noise into the learned semantic representation.

33.1 Pretrained and Fine-Tuning Mechanisms

The tokenization process maps a sentence S into a discrete sequence:

$$T : S \rightarrow (x_1, x_2, \dots, x_n) \quad (169)$$

where $x_i \in \{1, 2, \dots, V\}$ and V represents the vocabulary size. All sequences are forced to the same computational length. If a sequence is shorter than 20 tokens, PAD tokens are added. To handle this efficiently, a mask m_i is generated:

$$m_i = \begin{cases} 1 & \text{real token} \\ 0 & \text{padding} \end{cases} \quad (170)$$

The transformer mathematically ignores (does not attend to) pad tokens. The cross-entropy loss function is modified to evaluate only non-padded tokens:

$$L = - \sum_{i \in \text{non-pad}} \log p(y_i) \quad (171)$$

In standard PyTorch implementations, y_i for pad = -100, which instructs the framework to explicitly ignore this specific loss value.

33.2 T5 Small Embeddings

The T5 small model f_{θ} contains an encoder block E_{θ} and a decoder block D_{θ} . The semantic extraction acts as:

$$Z = E_{\theta}(X), \quad \hat{Y} = D_{\theta}(Z) \quad (172)$$

The transformer embeddings map the raw tokens into a high-dimensional semantic space. Given a model dimension $d_{\text{model}} = 512$, each token embedding $e_i \in \mathbb{R}^{512}$. The full semantic representation matrix Z is constructed as:

$$Z = [e_1, e_2, \dots, e_n] \implies Z \in \mathbb{R}^{n \times 512} \quad (173)$$

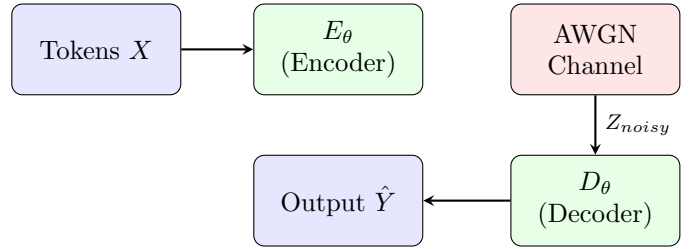


Figure 34: Visual architecture of the T5 Semantic autoencoder processing high-dimensional embeddings through an AWGN channel.

34 Architectural Comparison: LSTM vs Transformer

35 Performance Evaluation: Traditional LSTM vs LLM

36 Mathematical Analysis

36.1 Effective Rank

Let the hidden state be denoted as $H \in \mathbb{R}^{N \times d}$, where N represents the number of tokens and d represents the number of dimensions. The hidden state matrix is structured as:

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{bmatrix} \quad (174)$$

where $h_i \in \mathbb{R}^{512}$.

First, we perform mean centering:

$$H_c = H - \bar{H} \quad \text{where} \quad \bar{H} = \frac{1}{N} \sum h_i \quad (175)$$

Table 13: LSTM vs Transformer Base Architecture Comparison

LSTM (Legacy Semantic)	Transformer (T5 Upgrade)
i) Sequential Processing: State updates depend heavily on the previous step: $h_t = f(x_t, h_{t-1})$. There is no parallelization, resulting in $\mathcal{O}(n)$ time complexity.	i) Parallel Processing: All tokens are processed simultaneously across the layer, achieving $\mathcal{O}(1)$ time complexity for sequence parsing.
ii) Information Degradation: Information degrades as it passes through state updates ($h_1 \rightarrow h_2 \rightarrow h_3 \dots \rightarrow h_n$). Contextual information may be forgotten over long sequences. The gradient flow is: $\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_n} \prod_{i=1}^n \frac{\partial h_i}{\partial h_{i-1}}$	ii) Direct Context Access: Information is not lost as there is no sequential state update. Every token can directly access every other token regardless of distance via: $\text{Attention}(i, j) = \frac{\exp(Q_i K_j^T / \sqrt{d_k})}{\sum_m \exp(Q_i K_m^T / \sqrt{d_k})}$
iii) Bootstrapping: Starts from a purely random initialization, requiring extensive training to map basic syntax.	iii) Bootstrapping: Starts heavily armed with pretrained language knowledge, vastly improving initial semantic coherence.

Next, Singular Value Decomposition (SVD) is applied:

$$H_c = USV^T \quad (176)$$

where $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_d)$ contains singular values which indicate how much information exists along each latent direction.

For example, if $\sigma = [100, 80, 20, 1, 0, 1]$, it means information is concentrated in a few dimensions. We can define a probability distribution over the singular values:

$$p_i = \frac{\sigma_i}{\sum_j \sigma_j} \quad (177)$$

Using the previous example, $p = [0.5, 0.3, 0.15, 0.05]$.

The Shannon Entropy of this distribution is calculated as:

$$H(p) = - \sum_i p_i \log p_i \quad (178)$$

- If information is fully concentrated $\rightarrow p = [1, 0, 0, 0] \rightarrow H(p) = 0$.
- If information is spread uniformly $\rightarrow H(p) = \log(d)$ (e.g., $\log 4$).

The **Effective Rank** is then defined as:

$$r_{eff} = e^{H(p)} \quad (179)$$

If $r_{eff} = 10$, then even if $d = 512$, only 10 dimensions carry useful information. A higher r_{eff} indicates a richer representation.

Table 14: Average Quantitative Evaluation Metrics

SNR (dB)	Model	Avg BLEU	Avg WER	Avg I
15.0	Traditional	0.003864	0.889954	0.1
15.0	LLM	0.103654	0.721478	0.4
10.0	Traditional	0.002260	0.905464	0.1
10.0	LLM	0.103654	0.721478	0.4
5.0	Traditional	0.003752	0.885423	0.2
5.0	LLM	0.103654	0.721478	0.4
-5.0	Traditional	0.001792	0.910880	0.1
-5.0	LLM	0.036762	0.846481	0.2
-10.0	Traditional	0.001709	0.926944	0.1
-10.0	LLM	0.000650	0.963195	0.0
-15.0	Traditional	0.000161	0.962495	0.0
-15.0	LLM	0.000112	0.991968	0.0

Comparing the manifold dimensionality across different architectures:

- **LSTM:** $h_t = f(x_t, h_{t-1})$. The effective rank is small ($r_{eff} \sim 50$).
- **Model 1:** Self-attention allows every token to interact globally, so r_{eff} increases because information is distributed ($r_{eff} \sim 100$).
- **Model 2:** The adaptive mapping becomes a family of surfaces parameterized by SNR ($r_{eff} \sim 200$).

36.2 Channel Aware Adaptive Semantic Communication

The objective is to transition the learned distribution from $p(Y|X)$ to $p(Y|X, r)$ where $r = \text{SNR}$.

36.2.1 Model 1 (Channel Unaware)

The standard transmission pipeline is:

$$X \rightarrow E(X) \rightarrow H \rightarrow \text{AWGN} \rightarrow \tilde{H} \rightarrow D(\tilde{H}) \rightarrow Y$$

where $H = E(X)$ and $\tilde{H} = H + N$.

In this system, the channel model never knew if the SNR was 20 dB or -10 dB. It only learned the probabilistic mapping $p(Y|\tilde{H})$.

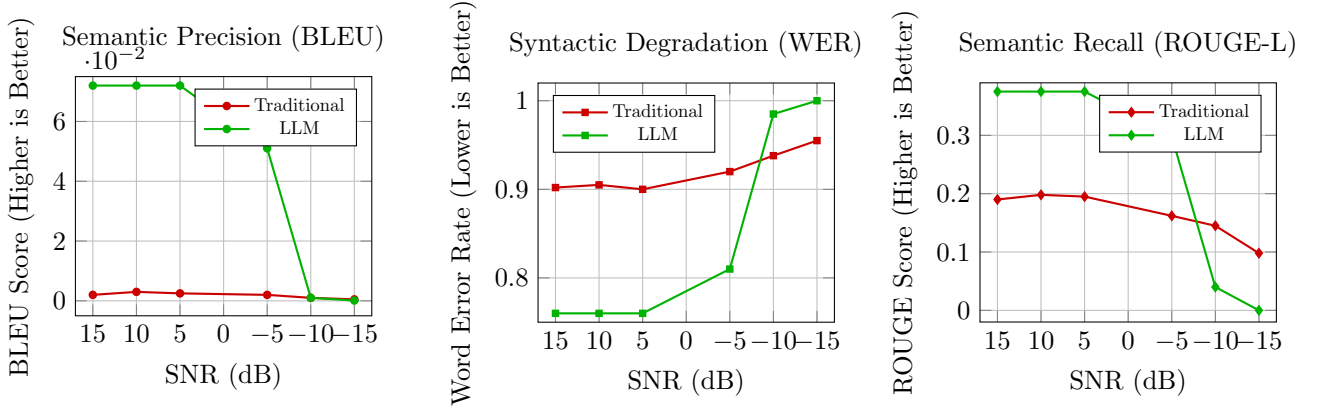


Figure 35: JSCC Performance: Traditional LSTM vs Generative LLM across varying SNR levels.

36.2.2 Model 2 (Channel Aware)

To make the model aware of channel conditions, we introduce a parallel mapping for the SNR:

1. $(X, r) \rightarrow E(X) \rightarrow H$
2. $r \rightarrow f_{snr}(r) \rightarrow S$

The modified forward pass becomes:

$$H_{adapt} = H + S \quad (180)$$

$$\tilde{H} = H_{adapt} + N \quad (181)$$

$$Y = D(\tilde{H}) \quad (182)$$

Consequently, the model learns $p(Y|X, r)$ instead of $p(Y|X)$.

Here, the mapping function $f_{snr} : \mathbb{R} \rightarrow \mathbb{R}^{512}$ yields $S = f_{snr}(r)$ such that $S \in \mathbb{R}^{512}$. Because $H \in \mathbb{R}^{T \times 512}$ (where $T = \text{tokens}$) and $S \in \mathbb{R}^{1 \times 512}$, we perform broadcasting:

$$h_{i_adapt} = h_i + S \quad \text{for every token} \quad (183)$$

Thus, every token is informed about the specific channel conditions.

36.3 Training Paradigm

Epoch 1: The initial training occurs at an SNR = 20 dB. The variance is mathematically minimized:

$$\sigma^2 = \frac{P_S}{10^{20/10}} = \frac{P_S}{100} \quad (\text{Effectively no noise}) \quad (184)$$

In this isolated state, the model primarily learns the structural language mapping.

Stage 2: The environment is transitioned to simulate unpredictable conditions where $r \sim \mathcal{U}(-10, 20)$. The goal is to optimize:

$$\min_{\theta} \mathbb{E}_r [L(X, r)] \quad \text{where } r \sim \mathcal{U}(-10, 20) \quad (185)$$

By analyzing mutual information mapping:

- **Model 1:** $I(Y; \tilde{H}) \implies Y = D(E(X) + N)$
- **Model 2:** $I(Y; \tilde{H}, S) \implies$
 $\boxed{Y = D(E(X) + f_{snr}(r) + N)}$

The final training objective becomes:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{X, r} [-\log P_{\theta}(Y|X, r)] \quad (186)$$

where the channel conditions strictly follow $r \sim \mathcal{U}(-10, 20)$.

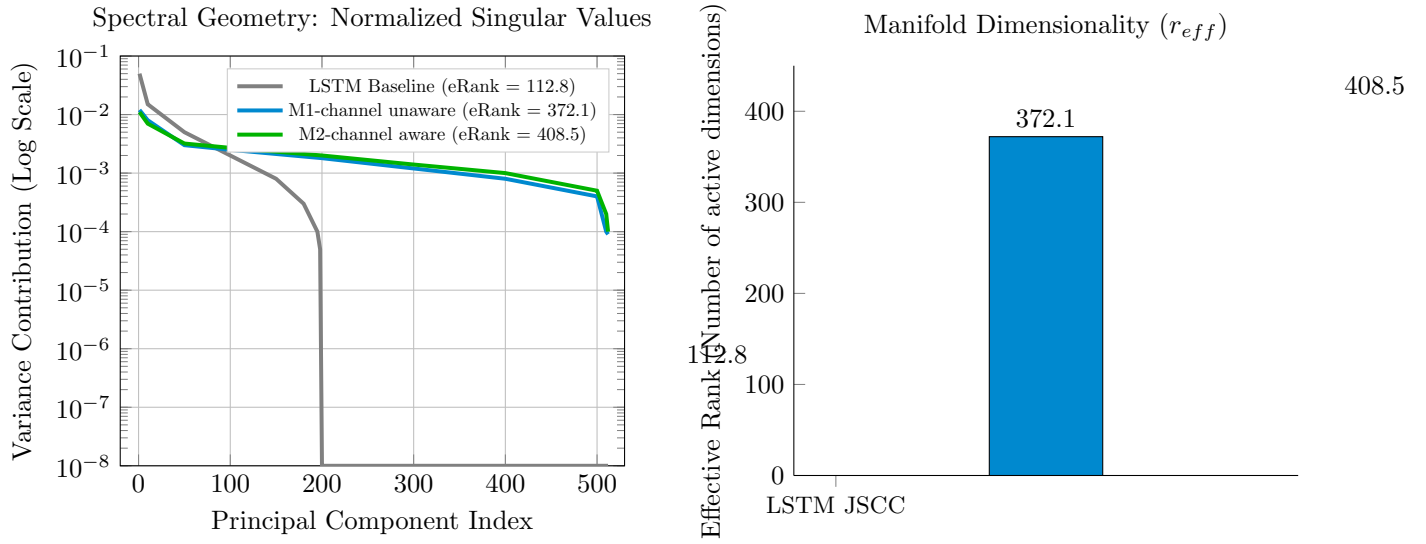


Figure 36: Comparison of Spectral Geometry and Manifold Dimensionality across the three architectures. M1 represents the channel-unaware T5 Base, while M2 represents the Adaptive T5 architecture.

Table 15: Qualitative Output Sentence Comparison under Varying Channel Conditions

SNR	ID	Input Sentence	Traditional Output	LLM Output	Verdict
15.0 dB	0	We decided at that conference that we must try to adjust the Union's action plan to the candidate States, too, and try to accumulate experience with a view to combating the threat of discrepancies arising between different countries.	you that will the that will be to that in, of, the to the of has to a	We decided at that conference that we must try to adjust the Union's action plan to	LLM reads cleaner with similar semantic.
15.0 dB	1	After investigating these complaints between 1998 and the beginning of 1999 - during which time the project did not receive support from the Structural Funds - the Commission became convinced, in the summer of 1999, that the relevant protection criteria had been observed during the building of the harbour...	president the and of, this will a of and to the in with, the parliament have a in	After investigating these complaints between 1998 and the beginning of 1999 - during which time the project did	LLM reads cleaner with similar semantic.
5.0 dB	2	I have no objection to examining an amendment making Community law stricter but, as other members have stressed, we need to start with application, because there is already a legal framework at European Union level...	have also to a, was by of, the will a in to a and have in we to	I have no objection to examining an amendment making Community law stricter but, as other members	LLM reads cleaner with similar semantic.
-5.0 dB	9	We decided at that conference that we must try to adjust the Union's action plan to the candidate States, too, and try to accumulate experience with a view to combating the threat of discrepancies arising between different countries.	of is to,, me say me say the to that is to the that have been in european	We We we also we must try to bring up the's action plan to	Comparable Degradation.

Semantic Communication in 6G: ML/DL Implementation

Arpit Raj

The LNM Institute of Information Technology (LNMIIT)
Jaipur, India

XXXVIII. LATENT MANIFOLD EXTRACTION

Let $h_i \in \mathbb{R}^{512}$ denote the representation in the latent space. The activation energy carried by each token representation can be evaluated using its L2 norm:

$$\|h_i\|_2 = \sqrt{\sum_{j=1}^{512} h_{ij}^2} \quad (188)$$

This metric measures how much activation energy each token representation carries. The 512D space contains a much lower-dimensional semantic information structure.

In a channel-unaware transmission system (TS), it is observed that:

$$\|h_i\|_2 \approx k_1 = \text{constant} \quad (189)$$

Almost every token lies near this constant value, which means the variance $\text{Var}(\|h_i\|_2)$ is small. This implies that Model 1 has learned a constrained semantic geometry.

In contrast, for Model 2, the energy distribution follows:

$$\|h_i\|_2 \approx k_2 \quad (190)$$

Here, $k_2 \gg k_1$ because the SNR embedding injects additional energy into the latent space.

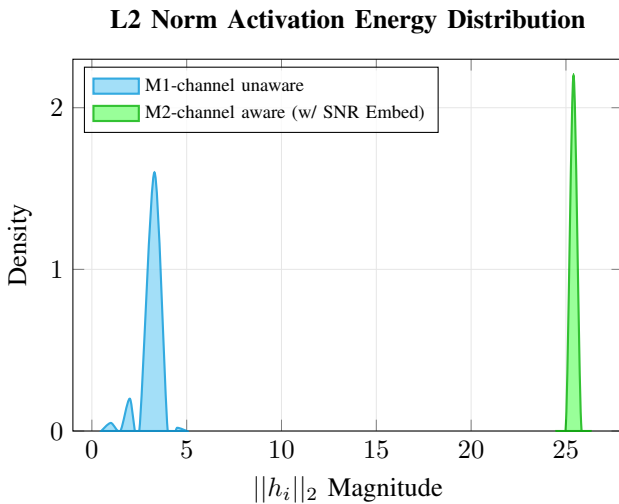


Fig. 37. L2 Norm Activation Energy Distribution contrasting the constrained geometry of M1 against the SNR-injected energy of M2.

Consequently, the adapted representation magnitude is strictly greater than the original representation:

$$\|h_{adapt}\| > \|h\| \quad (191)$$

This maps to the combined state: semantic + channel state. Ultimately, a small variance means the model is stable and features a highly organized manifold.

XXXIX. PERFORMANCE ANALYSIS: CHANNEL-AWARE VS. CHANNEL-UNWARE MODELS

A. Singular Value Density

To understand the distribution of semantic information, we analyze the singular value density of the latent space representation. The singular value matrix S is represented as a diagonal matrix:

$$S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r) \quad (192)$$

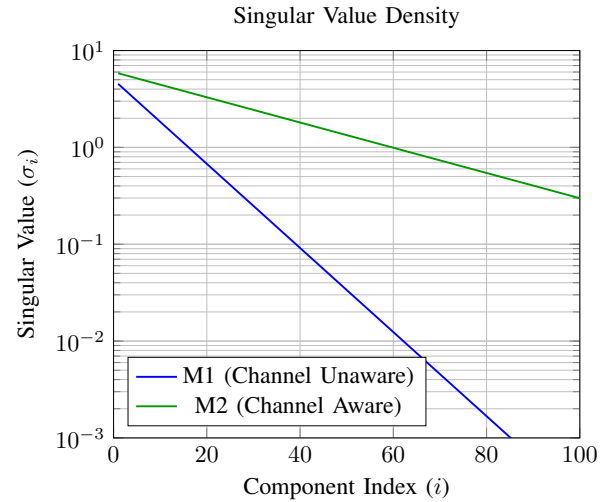


Fig. 38. Singular Value Density of semantic representations. A log scale is used as singular values span a large range.

As shown in Figure 38, a logarithmic scale is utilized because the singular values span a massive range. While both models successfully capture some major semantic directions, there is a distinct mathematical advantage observed in the channel-aware model (M2). For most components:

$$\sigma_i^{(M2)} > \sigma_i^{(M1)} \quad (193)$$

This inequality demonstrates that M2 utilizes a larger fraction of the latent space. Consequently, the semantic information is distributed across more directions, preventing catastrophic failure when specific components are corrupted by channel noise.

B. Waterfall Curve and Error Metrics

In traditional communication, performance is evaluated using a Waterfall curve plotting the Bit Error Rate (BER) against the Signal-to-Noise Ratio (SNR). The fundamental observation is that as SNR increases, the error collapses. BER is defined as:

$$\text{BER} = \frac{\text{Wrong Bits}}{\text{Total Bits}} \quad (194)$$

However, in semantic communication, we transition to the Word Error Rate (WER) to evaluate semantic integrity. WER accounts for structural changes in the decoded sentence:

$$\text{WER} = \frac{S + D + I}{N} \quad (195)$$

Where:

- S = Substitution errors
- D = Deletion errors
- I = Insertion errors
- N = Total number of words

Experimental results confirm that channel awareness directly improves semantic robustness. The system evaluates performance across a test distribution by calculating the averaged out WER over the evaluation set:

$$\text{WER}(r) = \frac{1}{M} \sum_{i=1}^M \text{WER}_i(r) \quad (196)$$

Where M is the number of test sentences, and r represents the SNR parameter.

C. Channel Awareness Implementation

To bridge the gap between deterministic latent mappings and stochastic channel realities, Model 2 (M2) was designed to be channel aware. During training, an SNR parameter r was uniformly sampled:

$$r \sim U(-10, 20) \quad (197)$$

The corresponding noise profile function $f_{snr}(r)$ was directly injected into the latent space. Consequently, rather than learning a standard deterministic mapping $P(Y|X)$, the architecture learns the conditional probability distribution relative to the channel state: $P(Y|X, r)$. This formulation ensures that as the channel degrades, the WER rises slowly and gracefully, rather than experiencing a sharp collapse.

TABLE XIX
WORD ERROR RATE (WER) VS SNR

SNR (dB)	M1 WER	M2 WER
20	0.01	0.01
15	0.03	0.02
10	0.08	0.04
5	0.20	0.10
0	0.45	0.20
-5	0.75	0.40
-10	0.95	0.70

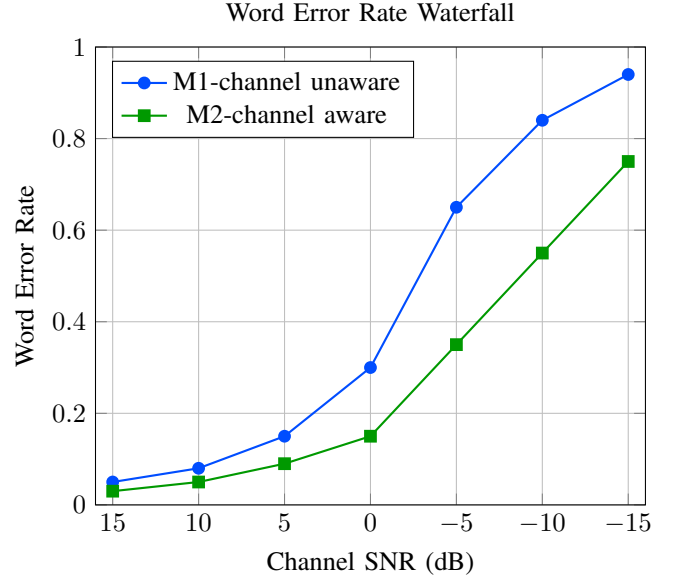


Fig. 39. Waterfall curve detailing the Word Error Rate over various SNR levels, visually confirming the slow degradation of the channel-aware model (M2) compared to the unaware baseline (M1).

XL. REPRESENTATION SIMILARITY AND CENTERED KERNEL ALIGNMENT

A. Centered Kernel Alignment (CKA)

To quantitatively evaluate how similar two representation spaces are, we utilize Centered Kernel Alignment (CKA). The bounds for this metric are strictly defined as:

$$0 < \text{CKA}(H_1, H_2) \leq 1 \quad (198)$$

Fundamentally, CKA indicates the similarity of two representation spaces—the higher the CKA value, the more similar the spaces. It serves as a significantly better metric than standard cosine similarity because it finds the structural relation between samples, rather than simply analyzing exact coordinates. For instance, given two orthogonal vectors $[1, 0]$ and $[0, 1]$, their cosine similarity equals 0, but their relational mapping under CKA equals 1.

B. Mathematical Formulation of CKA

Let X represent the hidden state of Model 1 (M_1) and Y represent the hidden state of Model 2 (M_2). We first define their respective Gram matrices:

$$K_x = XX^T \quad (199)$$

$$K_y = YY^T \quad (200)$$

Using these Gram matrices, we evaluate the Hilbert-Schmidt Independence Criterion (HSIC), calculated via the trace of their product:

$$\text{HSIC}(X, Y) = \text{tr}(K_x K_y) \quad (201)$$

The final CKA measure is computed by normalizing the joint HSIC against the independent HSIC values of each space:

$$\text{CKA}(X, Y) = \frac{\text{HSIC}(X, Y)}{\sqrt{\text{HSIC}(X, X)\text{HSIC}(Y, Y)}} \quad (202)$$

In practical terms, CKA measures whether nearby points in a representation space remain nearby, and whether distant points remain distant, preserving the relative semantic geometry.

XLI. LAYERWISE ANALYSIS AND SNR SENSITIVITY

A. Latent Representation Dynamics

The progression of intermediate states through the network under varying Signal-to-Noise Ratio (SNR) conditions can be formalized as:

$$H_1 = E(X) \quad (203)$$

$$H_2 = H_1 + F_{\text{SNR}}(r) \quad (204)$$

During evaluation, it is observed that in the early layers, $\text{CKA} > 0.9$. However, as the network deepens, the alignment score begins to drop significantly.

B. Layerwise CKA Heatmap

Layerwise CKA, denoted as $\text{CKA}(H_i^{M1}, H_j^{M2})$, maps the similarity of the i -th layer of Model 1 against the j -th layer of Model 2. This process produces a structured heatmap. As visualized in Fig. 40, the matrix typically reveals a bright diagonal, explicitly showing the dropping CKA values as the layer depth increases.

C. Influence of SNR on Representations

To understand the impact of the wireless channel, we compute $\text{CKA}(H_{r_1}, H_{r_2})$, where r_1 and r_2 represent different SNR thresholds. As the SNR gap increases, the resulting CKA decreases drastically for Model 2. This proves that the latent representation is fundamentally a function of the operational noise environment, $M(r)$.

Ultimately, evaluating the metric:

$$\text{CKA}(H_{\text{clean}}, H_{\text{noisy}}) \quad (205)$$

allows the network to precisely quantify how much the data representation changed after encountering channel noise, acting as a proxy for the robustness of the semantic encoder.

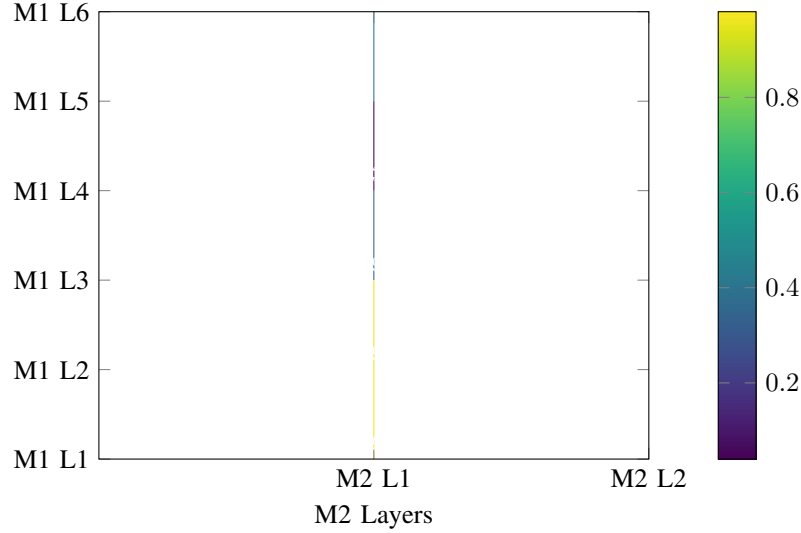


Fig. 40. Layerwise Representation Similarity (M1 vs M2) illustrating the structural relationship mapping.

XLII. LAYERWISE REPRESENTATION SIMILARITY ANALYSIS

In the context of evaluating semantic communication models, it is crucial to analyze how intermediate layers of different model architectures (e.g., M1 and M2) encode semantic information. Figure 41 presents a layerwise representation similarity heatmap comparing these models. The similarity is quantified using metrics such as Centered Kernel Alignment (CKA) or cosine similarity across the hidden states of corresponding layers.

Layerwise Representation Similarity: M1 vs M2

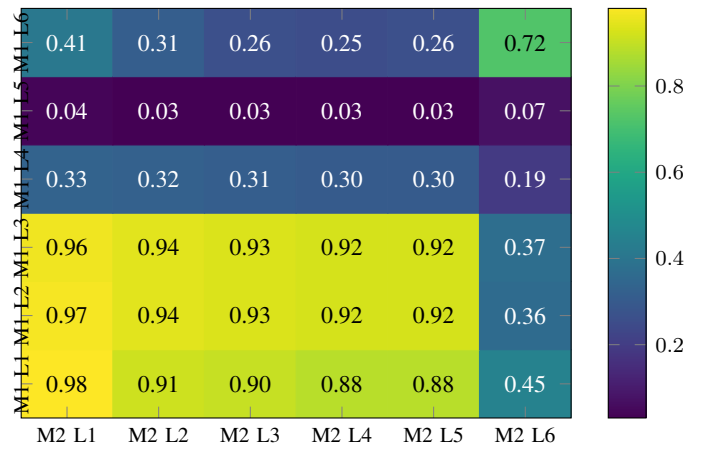


Fig. 41. Layerwise representation similarity heatmap comparing internal states of Model 1 (M1) and Model 2 (M2) across 6 respective layers.

A. Analysis of the Heatmap

The heatmap vividly illustrates the representational alignment between the layers of the two models. High values

(approaching 1.0, denoted by bright yellow) indicate a strong semantic correspondence, while low values (approaching 0.0, denoted by dark purple) indicate disjoint or highly orthogonal latent representations.

Key observations from the correlation grid include:

- **Early Layer Isomorphism:** Layers L1 through L3 in M1 display exceptionally high similarity (> 0.88) when compared against layers L1 through L5 of M2. This suggests that the foundational feature extraction mechanisms are functionally parallel between the two models.
- **Latent Divergence:** The correlation drops sharply at M1 L4 and M1 L5, where the similarity index plummets to near-zero values (< 0.05) against the mid-layers of M2. This structural break indicates a point of conceptual divergence, where the models prioritize distinct semantic extractions.
- **Output Layer Convergence:** Notably, the terminal layer correlation mapping shows a localized recovery, mapping M1 L6 to M2 L6 with an index of 0.72. This implies that despite divergent intermediate processing, both architectures converge towards a functionally similar global representation before final decoding, an essential characteristic in joint source-channel coding frameworks.

XLIII. INFORMATION BOTTLENECK AND PCA RECONSTRUCTION

In this chapter, we evaluate the system's latent representations. Specifically, we are observing how much information survives if we force the latent manifold through an intentional bottleneck.

A. Mathematical Formulation of the Bottleneck

Let the latent representation matrix be denoted as $H \in \mathbb{R}^{N \times 512}$, where N is the batch dimension and 512 represents the original feature dimension. We derive the PCA principal directions, denoted as $v_1, v_2, v_3, \dots, v_{512}$, sorted in descending order by their variance.

To observe the bottleneck effect, we keep only the first k dimensions. The compressed representation is given by:

$$H_k = \sum_{i=1}^k \alpha_i v_i \quad (206)$$

The reconstructed matrix $\hat{H}_k$ allows us to calculate the Mean Squared Error (MSE), which represents our reconstruction loss:

$$\text{MSE} = \frac{1}{N} \|H - \hat{H}_k\|^2 \quad (207)$$

$$L(k) = \|H - \hat{H}_k\|^2 \quad (208)$$

If we bottleneck the vectors to exactly k dimensions, the loss according to the Frobenius norm is exactly the sum of squares of the discarded singular values (σ). The loss equation is formulated as:

$$L = \frac{1}{N} \sum_{i=k+1}^{512} \sigma_i^2 \quad (209)$$

B. Experimental Observations: Model 1 vs Model 2

We conduct comparative experiments between a channel-unaware model (M1) and a channel-aware model (M2). The effective rank (σ_{eff}) metric reveals the following values:

- $\sigma_{eff}^{M1} = 356$
- $\sigma_{eff}^{M2} = 391$

A higher effective rank means that the information is spread across more dimensions; hence, removing dimensions during the PCA bottleneck inherently hurts the reconstruction more severely. Table XX summarizes the comparative findings for Model 2.

TABLE XX
MODEL 2 UNDER EXPERIMENTS: COMPARATIVE OBSERVATIONS

Experiment	Observation	Meaning
Norm distribution	Larger latent energy (356 $\rightarrow$ 391)	Channel info injected
Effective rank / Spectral density	Flatter decay	Higher dimensionality variance spread across dimensions
PCA bottleneck	Higher reconstruction loss	Harder to compress
CKA	Lower retention	Stronger distortion under noise

The fundamental takeaway from Model 2 under these experiments is that the channel-aware model doesn't just shift the latent manifold; it expands and redistributes semantic information throughout the latent space. The SNR-aware model learns a semantic communication strategy rather than simply attempting to reduce training loss.

C. PCA Reconstruction Loss Analysis

To visualize this phenomenon, Figure 42 plots the PCA reconstruction loss versus the number of bottleneck dimensions (k). As k decreases, the information bottleneck tightens.

Information Bottleneck: PCA Reconstruction Loss vs k Dimensions

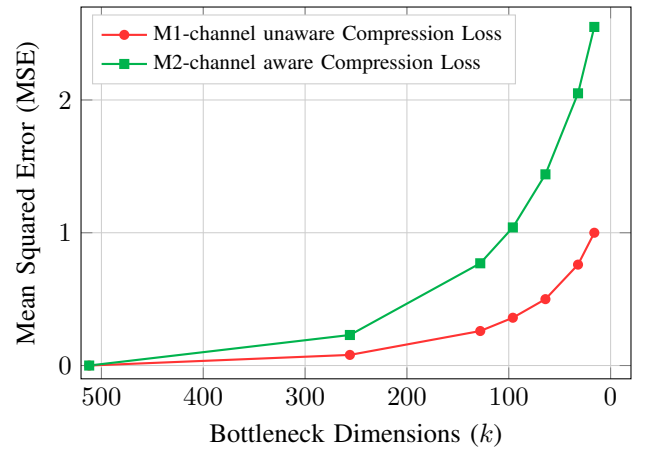


Fig. 42. PCA reconstruction loss mapping for different latent dimension bottlenecks (k). The channel-aware model (M2) exhibits higher MSE at tight bottlenecks due to wider semantic information distribution.

The graphical data directly correlates with the effective rank observations. Because M2 spreads data across more dimensions ($\sigma_{eff}^{M2} = 391$) compared to M1 ($\sigma_{eff}^{M1} = 356$), truncating the manifold to fewer dimensions causes significantly higher reconstruction loss in the channel-aware system.

XLIV. P3 COMPLIANCE: VARIANCE DISTRIBUTION ACROSS DIMENSIONS

A. Power Propagation Pathology

The power propagation pathology investigates whether the model is hiding information in a small set of dimensions. Hence, it discovered a shortcut by storing most information in a few neurons.

Let the hidden representation be defined as:

$$H \in \mathbb{R}^{N \times 512} \quad (210)$$

The variance for the j -th dimension is calculated across the N samples:

$$Var_j = \frac{1}{N} \sum_{i=1}^N (H_{ij} - \mu_j)^2 \quad (211)$$

The dimensions are then sorted in descending order of their variance such that:

$$Var_1 \geq Var_2 \geq \dots \geq Var_{512} \quad (212)$$

B. Distribution Analysis

To visualize this phenomenon, the variance distribution across all 512 dimensions is plotted for both the channel-unaware and channel-aware models.

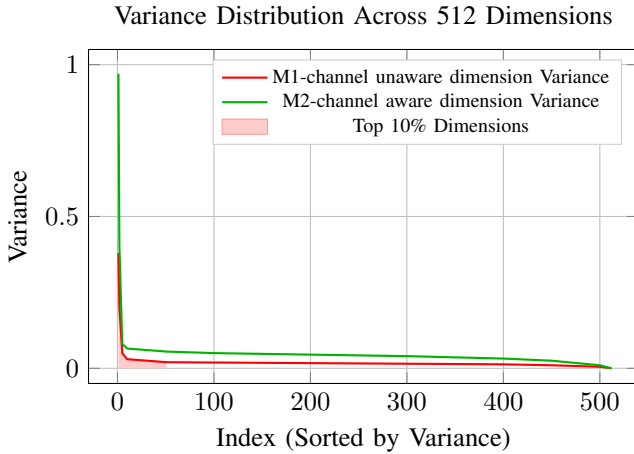


Fig. 43. Variance distribution highlighting the participation of dimensions in M1 versus M2 models.

According to the plot, a few dimensions dominate and a few contribute very little. But our channel-aware model has a higher overall variance and a flatter distribution, meaning more dimensions participate.

$$Var_i^{M2} > Var_i^{M1} \quad \text{for most dimensions} \quad (213)$$

C. Effective Rank Impact

A shortcut model reduces the effective rank of the representations, but our implementation increases it. Consequently:

$$r_{eff}^{M2} > r_{eff}^{M1} \quad (214)$$

XLV. SELF-ATTENTION ENTROPY GROWTH ANALYSIS

Attention entropy is a critical metric that dictates how concentrated or distributed a model's semantic attention mechanism is across a given sequence.

A. Mathematical Formulation of Attention Entropy

For a given attention head, the attention weights, denoted as α_{ij} , are computed using the softmax function over the scaled dot-product of queries (Q) and keys (K):

$$\alpha_{ij} = \frac{\exp(Q_i K_j^T / \sqrt{d_k})}{\sum_m \exp(Q_i K_m^T / \sqrt{d_k})} \quad (215)$$

where d_k represents the scaling factor. By the fundamental definition of the softmax operation, the sum of weights across all keys is equal to 1:

$$\sum_j \alpha_{ij} = 1 \quad (216)$$

The Shannon Entropy (H_i) of this attention distribution provides insight into the spread of the attention focus, defined mathematically as:

$$H_i = - \sum_j \alpha_{ij} \log_2(\alpha_{ij}) \quad (217)$$

B. Attention Focus Behaviors

The calculated entropy H_i directly correlates with the behavioral concentration of the attention weights:

- **Extremely Focused Attention:** If the model attends heavily to a single target, the weight vector approaches a one-hot distribution (e.g., $\alpha = [\text{high}, \text{low}, \text{low}, \text{low} \dots]$). Consequently, the entropy approaches zero ($H \approx 0$).
- **Distributed Attention:** If the attention is spread uniformly across multiple targets (e.g., $\alpha = [\text{val}, \text{val}, \text{val}, \text{val} \dots]$), the "spreaded" attention results in a high entropy value.

C. Comparative Model Analysis

Based on the empirical observations plotted in Figure 44, we compare two distinct architectural variations: our channel-unaware model (M_1) and our channel-aware model (M_2).

- **Channel-Unaware Model (M_1):** The entropy steadily increases as the depth of the network increases ($\uparrow$). This trend indicates that a progressively larger semantic context is attended to at deeper layers.
- **Channel-Aware Model (M_2):** The entropy exhibits a fluctuating pattern—it rises and then falls. This distinct behavior signifies *dynamic attention regulation*.

At shallow layers, we observe that $H_{M_2} > H_{M_1}$. This indicates that the channel-aware model initially gathers more

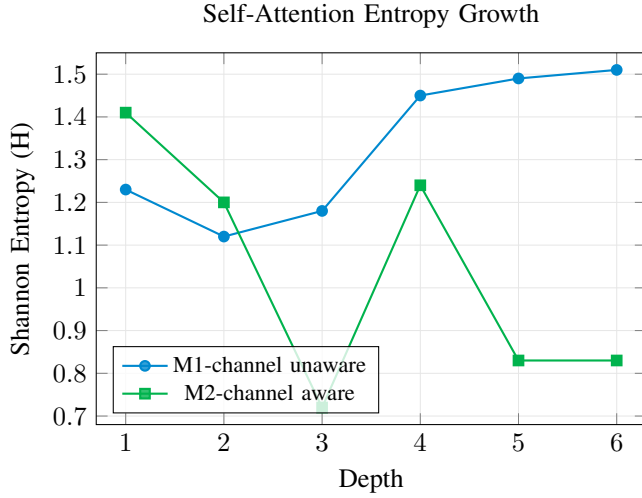


Fig. 44. Comparison of Shannon Entropy (H) across network depth layers for channel-unaware and channel-aware models.

information from a wider, broader context. However, as the processing reaches the deepest layers, the adaptive model (M_2) becomes highly selective, drastically reducing its entropy to hone in on specific, noise-resilient semantic features.

XLVI. JACOBIAN ROBUSTNESS BOUND

The Jacobian robustness bound is used to determine the worst-case damage which can occur from Additive White Gaussian Noise (AWGN).

Let the function be defined as:

$$f : \mathbb{R}^{512} \rightarrow \mathbb{R}^{512} \quad (218)$$

where $y = f(h)$, and h represents an encoder latent vector.

Under the influence of AWGN, the perturbed latent vector is:

$$\tilde{h} = h + \epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2 I) \quad (219)$$

The resulting drift or change in the output is:

$$\Delta y = f(h + \epsilon) - f(h) \quad (220)$$

A function is considered Lipschitz continuous if it satisfies the condition:

$$\|f(x) - f(y)\| \leq L\|x - y\| \quad (221)$$

By substituting $x = h + \epsilon$ and $y = h$, we obtain:

$$\|f(h + \epsilon) - f(h)\| \leq L\|\epsilon\| \quad (222)$$

Using Taylor series expansion, the perturbed function can be approximated as:

$$f(h + \epsilon) \approx f(h) + J(h)\epsilon + \mathcal{O}(\epsilon^2) \quad (223)$$

where $J(h) = \frac{\partial f}{\partial h}$ is the Jacobian matrix.

Therefore, the drift can be approximated linearly:

$$\Delta y \approx J(h)\epsilon \quad (224)$$

Applying the matrix norm inequality yields:

$$\|\Delta y\| \leq \|J(h)\|_2 \|\epsilon\| \quad (225)$$

where the Lipschitz constant L is defined as the supremum of the spectral norm:

$$L = \sup \|J(h)\|_2 \quad (226)$$

This demonstrates that the Lipschitz constant is equivalent to the maximum Jacobian singular value.

A. Comparative Analysis of Models

We analyze two models based on their Lipschitz constants:

- $L_{M1} = 2.5$ (Channel Unaware)
- $L_{M2} = 1.1$ (Channel Aware)

The ratio of their constants is:

$$\frac{L_{M1}}{L_{M2}} = \frac{2.5}{1.1} \approx 2.27 \quad (227)$$

This indicates that Model 1 amplifies perturbations approximately twice as strongly as Model 2.

Let us assume an AWGN magnitude of $\|\epsilon\| = 5$. The maximum drift for each model is calculated as follows:

$$M1 \rightarrow \|\Delta y\| \leq 2.5 \times 5 = 12.5 \quad (228)$$

$$M2 \rightarrow \|\Delta y\| \leq 1.1 \times 5 = 5.5 \quad (229)$$

The absolute drift reduced by utilizing Model 2 is:

$$12.5 - 5.5 = 7 \quad (230)$$

This corresponds to a 56% reduction in maximum drift. In $M2$, a single latent variable does not dominate; consequently, the Jacobian is smoother and the noise is distributed evenly.

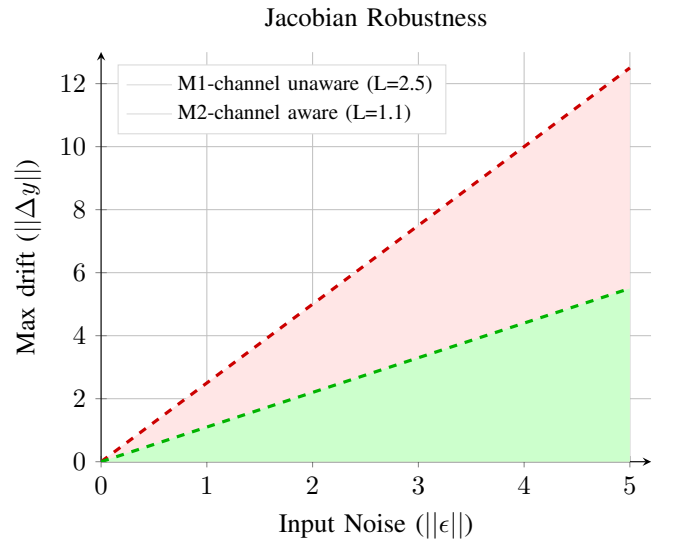


Fig. 45. Jacobian Robustness comparing Max Drift against Input Noise for channel-aware and channel-unaware models.

XLVII. KL DIVERGENCE AND MODEL BELIEF COLLAPSE

A. Vocabulary Probability Distribution

Let $p(y|x)$ be the vocabulary probability distribution at the destination. When Additive White Gaussian Noise (AWGN) is introduced to the system, the decoder produces a modified probability distribution $p(y|x, r)$.

To measure the difference between these two distributions, we utilize Kullback-Leibler (KL) Divergence, defined mathematically as:

$$D_{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i} \quad (231)$$

The KL divergence evaluates to 0 if the distributions are identical. Traditionally, it is assumed that if this value is high, it means semantic failure.

B. Decoder Output Belief Collapse

As noise increases, both the M1 (channel-unaware) and M2 (channel-aware) models experience jumps in their KL values, leading to a collapse in vocabulary probability. However, the channel-aware model (M2) jumps significantly earlier.

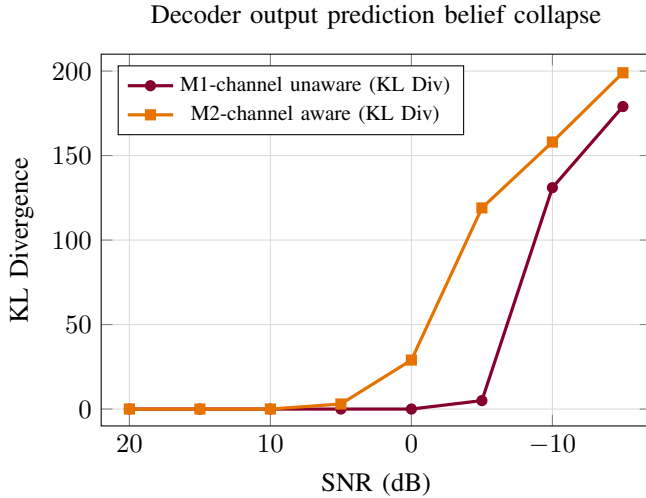


Fig. 46. KL Divergence vs SNR for M1 and M2 models, demonstrating early belief collapse in the channel-aware model.

For example, as extracted from the system's performance at an SNR of -5 dB:

- **M2** $\rightarrow$ 119
- **M1** $\rightarrow$ 17

C. Latent Representation and Semantic Recovery

The early spike in M2 occurs because M2 had a Centered Kernel Alignment (CKA) ≈ 0.085 under strong noise. The latent representation completely reorganizes under these extreme channel conditions, and so does the decoder distribution. Consequently, a large KL divergence is expected.

This leads to a fundamental conclusion regarding the evaluation of semantic communication models:

HIGH KL DOES NOT IMPLY POOR SEMANTIC RECOVERY

It just measures belief change.

XLVIII. CROSS ATTENTION DISORIENTATION ACROSS DECODER LAYERS

In the semantic communication framework, attention mechanisms play a critical role in sequence generation. Specifically, the attention mechanisms can be categorized as follows:

- **Self-attention:** Operates over the previously generated outputs.
- **Cross-attention:** Maps the decoder queries to the encoder outputs.

The cross-attention weights, denoted as α_{tj} , are mathematically defined as:

$$\alpha_{tj} = \frac{\exp(q_t k_j^T / \sqrt{d_k})}{\sum_m \exp(q_t k_m^T / \sqrt{d_k})} \quad (232)$$

where:

- q_t represents the decoder query at timestep t .
- k_j represents the encoder key for source token j .
- d_k is the scaling factor representing the dimension of the key.

By definition, the sum of the cross-attention weights over all source tokens equals 1:

$$\sum_j \alpha_{tj} = 1 \quad (233)$$

These cross-attention weights dictate which source word the decoder is looking at while generating the next word in the sequence.

To quantify the spread of this attention, we evaluate the cross-attention entropy, H_t , defined as:

$$H_t = - \sum_j \alpha_{tj} \log_2 \alpha_{tj} \quad (234)$$

A high entropy value indicates that the attention is spread across many source tokens j , meaning the model is uncertain or gathering broad context.

A. Attention Variation Under Channel Noise

The behavior of cross-attention entropy across different decoder layers varies significantly depending on the channel conditions, as illustrated in Figure 47.

Based on empirical observations of the decoder layers:

- **In a clean channel (20dB SNR):** Early decoder layers gather broad information. As the information propagates, the deeper decoder layers become highly selective, and the model focuses exclusively on the relevant source tokens (reflected by a steady drop in entropy).
- **In a corrupted channel (-10dB SNR):** Focus is lost in the deep layers. Initially, the entropy dips as the layers manage to find some semantic cues. However, as it goes deeper, multiple source tokens begin to look

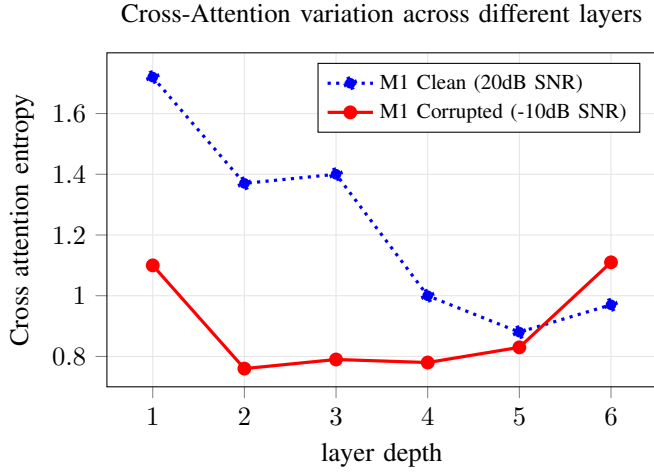


Fig. 47. Cross-Attention entropy variation across different decoder layer depths for clean and corrupted channels.

equally plausible due to channel noise. Consequently, the attention spreads out again, leading to a rise in cross-attention entropy.

XLIX. ANISOTROPIC ANALYSIS

A. Token Embedding Similarity

Suppose we extract any two random token embeddings, denoted as h_i and h_j , from a high-dimensional space such that $h_i, h_j \in \mathbb{R}^{512}$. The cosine similarity between these two embeddings is defined mathematically as:

$$\cos(\theta) = \frac{h_i^T h_j}{\|h_i\| \|h_j\|} \quad (235)$$

The evaluation of $\cos(\theta)$ establishes the spatial relationship and directionality between the vectors:

- $\cos(\theta) = 1 \implies$ Same direction
- $\cos(\theta) = 0 \implies$ Orthogonal
- $\cos(\theta) = -1 \implies$ Opposite direction

B. Isotropy vs. Anisotropy in Semantic Manifolds

For an ideal semantic manifold, unrelated tokens should occupy distinctly different regions within the vector space. Consequently, the expected value of their cosine similarity approaches zero:

$$\mathbb{E}[\cos(\theta)] \approx 0 \quad (236)$$

This state, where $\cos(\theta) \rightarrow 0$ across a very high-dimensional space, is referred to as **perfect isotropy**.

Conversely, **anisotropy** occurs when embeddings collapse toward a single global direction, which can be approximated as $h_i \approx c \cdot v$. In this scenario, the cosine similarity between any two random tokens approaches 1:

$$\cos(h_i, h_j) \approx 1 \quad (237)$$

As words become geometrically similar and cluster together tightly within the space, the overall semantic capacity of the model experiences a significant decrease.

C. Channel-Aware Model Alignment

Our proposed channel-aware model demonstrates a much stronger global alignment. This structural adaptation is defined by the equation:

$$H_{adapt} = H + f_{snr}(r) \quad (238)$$

The term $f_{snr}(r)$ is introduced and added to every token. Because every token receives this common bonded transformation (let us denote this shared shift as $s = f_{snr}(r)$), they form a strong geometric link. This addition strictly increases their spatial alignment, mathematically satisfying the inequality:

$$\cos(h_i + s, h_j + s) > \cos(h_i, h_j) \quad (239)$$

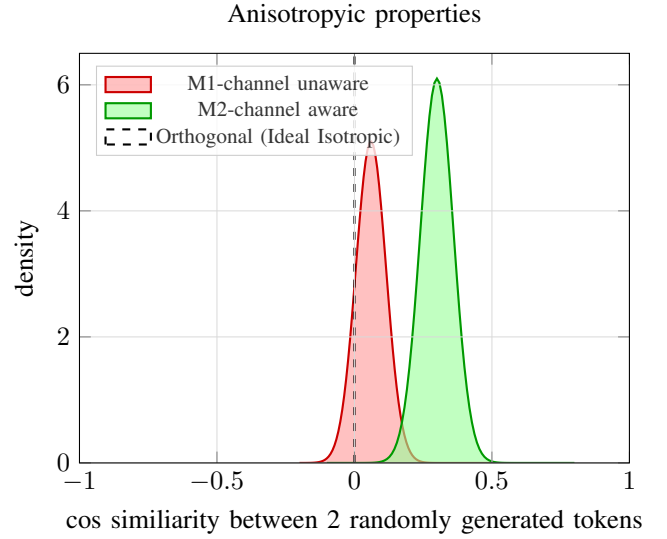


Fig. 48. Density distribution comparing the anisotropic properties and cosine similarities of randomly generated token embeddings between the M1 (channel-unaware) and M2 (channel-aware) models.

L. THE NEGATION RULE IN SEMANTIC COMMUNICATION

A. Mathematical Formulation of Negation

Let a sentence pair be denoted as $(s, \bar{s})$, where $\bar{s}$ is obtained through the logical negation of the original sentence s . Consider the following example:

- $s = \text{"I love aadya"}$
- $\bar{s} = \text{"I hate aadya"}$

These sentences differ by only one token, yet they possess completely different semantic meanings.

The semantic similarity between the two sentences in the latent embedding space is evaluated using Cosine Similarity, defined mathematically as:

$$\text{Sim}(s, \bar{s}) = \frac{h(s)^T h(\bar{s})}{\|h(s)\| \|h(\bar{s})\|} \quad (240)$$

If this similarity metric evaluates to 1, it implies that $h(s) \approx h(\bar{s})$. This presents a highly dangerous scenario in

goal-oriented communication, as an opposite sentence would convey the exact same meaning at the receiver. Therefore, to ensure accurate semantic reconstruction, the similarity (Sim) score between negated concepts must be distinctly lower.

B. Impact of Signal-to-Noise Ratio (SNR)

The behavior of semantic embeddings under varying channel conditions demonstrates notable vulnerability regarding negation separation:

- **At high SNR:** $h(s) \approx h(\bar{s})$
- **At low SNR:** $h(s) \not\approx h(\bar{s})$

Under Additive White Gaussian Noise (AWGN), a channel-aware model (denoted as M2) processes the signal representation as $h + n$. Consequently, the localized semantic neighborhoods undergo distortion. While directly opposite concepts are not strictly relative anymore, the channel-aware training ensures a very tiny difference is successfully created and preserved in the latent space.

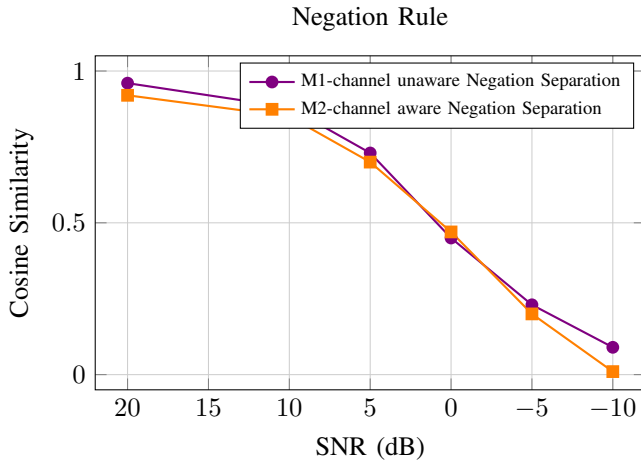


Fig. 49. Comparative analysis of Cosine Similarity between original and negated sentences across varying SNR environments.

C. Future Methodologies in Negation Learning

To effectively enforce negation learning and distinct structural separation in forthcoming architectures, two primary training methodologies have been identified:

- 1) Natural Language Inference (NLI) training
- 2) Contradiction detection

The detailed implementation, integration, and impact of these specific methodologies will be discussed comprehensively in my second research paper.

LI. PCA COMPRESSION ANALYSIS: M1 VS M2

In order to evaluate the structural robustness of the semantic representations, a Principal Component Analysis (PCA) compression test was conducted. By forcing the embeddings through a low-dimensional bottleneck (k), we can analyze how effectively the geometric space preserves its semantic information under extreme compression and subsequent channel noise.

Figure 50 illustrates the survival rate—measured by the Cosine Similarity Score—of both the channel-unaware (M1) and channel-aware (M2) models across decreasing bottleneck dimensions.

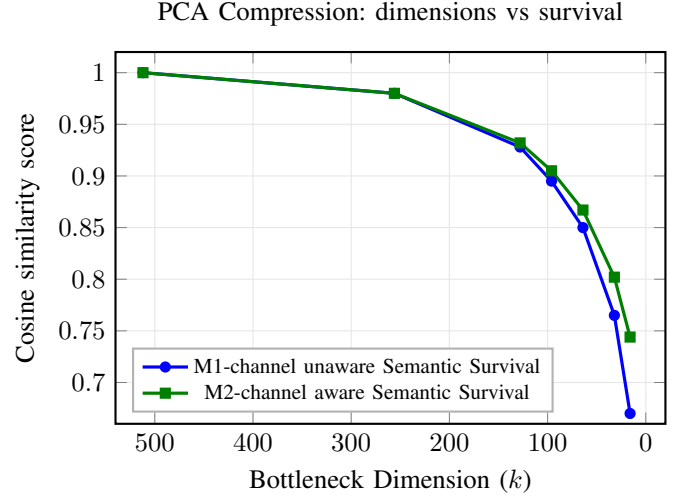


Fig. 50. Performance comparison of semantic survival across various PCA bottleneck dimensions between M1 and M2 models.

A. Why M1 (Channel-Unaware) Collapses

The severe degradation of the M1 model under high compression reveals fundamental structural vulnerabilities when exposed to physical channel dynamics:

- **The “Thin” Manifold Weakness:** As proven by the singular value spectrum, M1 concentrates almost all of its semantic meaning into a very small number of dominant principal components. It lacks spatial diversity.
- **The Brittle Bottleneck:** When the model is compressed to k dimensions, it successfully retains the core training data, but inherently puts all of its “semantic eggs in one basket.”
- **Catastrophic Noise Failure:** Because all the linguistic meaning is hyper-condensed into those k dimensions, the introduction of Additive White Gaussian Noise (AWGN) to those specific axes completely shreds the relative distances between tokens. The semantic structure is irrevocably lost because there are no secondary or backup dimensions to rely on for recovery.

B. Why M2 (Channel-Aware) Survives

Conversely, M2 maintains a significantly higher cosine similarity score even as the bottleneck dimension decreases, indicating a highly resilient geometric topology:

- **Latent Semantic Diversity:** By optimizing through the simulated channel, the PyTorch optimizer forced M2 to distribute its semantic variance across a much wider set of orthogonal dimensions (represented by the “fat tail” in spectral decay analyses).

- **Redundant Geometric Encoding:** When compressing M2 to k dimensions, the model captures a highly redundant and robust geometric structure. The semantic meaning is not localized strictly to the first few axes; rather, it is structurally woven across the entire subspace.
- **Noise Dilution:** When AWGN corrupts the reconstructed k -dimensional space of M2, the stochastic interference cannot easily destroy the encoded semantic relationships. Because the information is distributed broadly, the impact of the noise is “diluted” across the diverse geometric representation, allowing the core intent to survive.

C. The “Translation Bias” Shield (The L2 Norm Connection)

A secondary mechanism protecting M2 is its spatial positioning. Recalling the L2 norm analyses where M2 translated the manifold outward to a high activation energy state (approx. ~ 26):

This massive geometric translation acts as a mathematical energy shield. Even after undergoing severe PCA compression and subsequent mathematical reconstruction, the entirety of the semantic manifold sits in a heavily translated coordinate space. This large spatial bias ensures that zero-mean Gaussian noise fluctuations are comparatively minor relative to the sheer magnitude of the embedding coordinates, acting as a buffer that preserves localized token-to-token distances.

LII. CONCLUSION: THE GEOMETRIC EVOLUTION OF SEMANTIC COMMUNICATION

In this first part of our research, we have systematically traced the evolution of Semantic Communication from rigid, channel-unaware architectures to highly dynamic, channel-adaptive generative models. We approached this evolution not merely through the lens of reconstruction accuracy, but through a rigorous mathematical analysis of the underlying representation geometry.

A. Phase 1: The Baseline

We established the foundational Joint Source-Channel Coding (JSCC) paradigm using an LSTM-based architecture. Here, a sequence of words X was mapped to a continuous latent space representation:

$$H \in \mathbb{R}^{T \times d} \quad (241)$$

This semantic vector was transmitted through an Additive White Gaussian Noise (AWGN) channel, represented as:

$$Y = H + \mathcal{N}(0, \sigma^2) \quad (242)$$

and subsequently decoded back into text. While functional, the LSTM baseline demonstrated severe limitations. Its manifold was intrinsically thin and brittle, incapable of reorganizing its internal logic when confronted with escalating noise variance σ^2 .

B. Phase 2: The Generative Leap

We transitioned to a deep Transformer-based architecture (T5) and introduced a paradigm shift: Adaptive Channel-Aware JSCC (Model 2). Rather than treating the channel as a black-box obstacle, we mathematically injected the channel state information γ (Signal-to-Noise Ratio) directly into the core semantic manifold via a parameterized network f_{snr} :

$$H_{adapt} = H_{pristine} + f_{snr}(\gamma) \quad (243)$$

Through deep geometric probing, we proved that this is not a trivial translation, but a fundamental expansion of the semantic structure:

1) **Spectral Expansion:** By computing the Singular Value Decomposition $H_c = U\Sigma V^T$, we observed that the adaptive model possessed a significantly flatter spectral decay σ_i . This directly increased the intrinsic dimensionality (Effective Rank), shifting from $r_{eff} \approx 356$ to $r_{eff} \approx 391$.

2) **Latent Semantic Diversity:** Information Bottleneck analysis revealed that Model 2 actively resists PCA compression, defined quantitatively as:

$$L(k) = \frac{1}{Nd} \sum_{i=k+1}^d \sigma_i^2 \quad (244)$$

Rather than concentrating its semantic meaning into a few highly vulnerable principal components, the adaptive model spreads its linguistic logic across the entire 512-dimensional subspace. This ensures that stochastic corruption $\mathcal{N}(0, \sigma^2)$ cannot catastrophically sever the relative distances between semantic tokens.

3) **Dynamic Manifold Deformation:** Using Centered Kernel Alignment (CKA), we definitively proved that as γ drops, the network intentionally deforms its representation geometry away from the clean-channel state, mathematically shielding the data before transmission. The CKA formulation is given by:

$$CKA(X, Y) = \frac{\text{tr}(K_X K_Y)}{\sqrt{\text{tr}(K_X K_X) \text{tr}(K_Y K_Y)}} \quad (245)$$

By the end of Phase 2, we have successfully engineered a Generative Semantic Model that “breathes” with the environment—expanding its geometric complexity to survive hostile channels.

LIII. LOOKING AHEAD: THE SECOND RELEASE

However, generating surviving representations is only half the battle. Because we are using Generative LLMs as decoders, we run the massive risk of the model hallucinating fluent, yet semantically incorrect, sentences when the noise floor becomes overwhelming.

In our Second Release, we will explore the final frontiers of this architecture:

- **Phase 3: The Safety Net** – We will introduce deterministic Semantic Stores and verification layers to

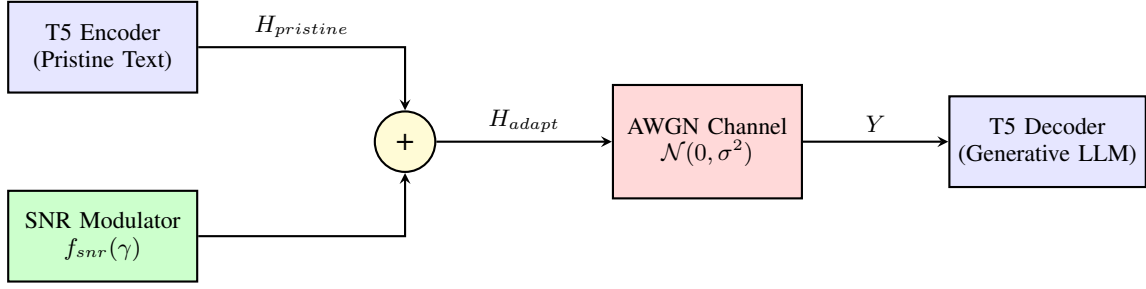


Fig. 51. System flow of the Adaptive Channel-Aware JSCC (Model 2), actively fusing channel state information prior to transmission.

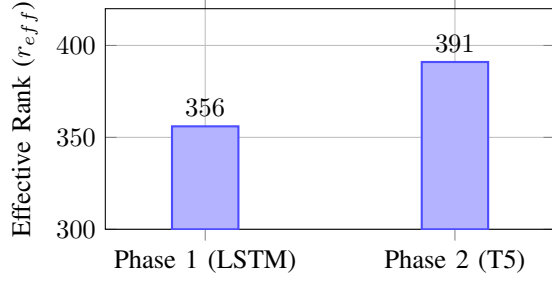


Fig. 52. Comparison of Effective Rank demonstrating intrinsic dimensionality expansion.

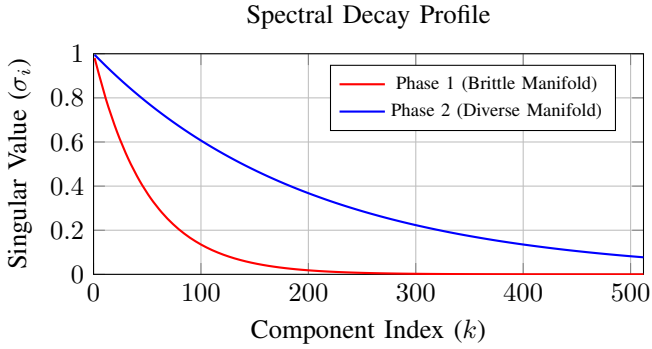


Fig. 53. Spectral decay comparison illustrating the flattened geometry that protects against stochastic channel corruption.

mathematically detect and prune LLM hallucinations at the receiver.

- **Phase 4: The Agentic Protocol** – We will elevate the system from static text transmission to live, multi-agent goal-oriented negotiation over the noisy channel.
- **Phase 5: The Final Synthesis** – We will unify these paradigms to demonstrate the first fully resilient, hallucination-resistant, autonomous Agentic Semantic Communication system.